

关于本书

《The Wayland Protocol》是一本关于 Wayland 显示服务器协议的免费在线书籍，覆盖核心协议、libwayland、表面（surface）、seat 输入处理、XDG shell、剪贴板与协议扩展等内容。

本仓库是它的中文译本，使用 [mdBook](#) 构建，可生成 HTML 网站与 PDF。

- 在线阅读：<https://bitristan.github.io/the-wayland-protocol-cn/>
- PDF: [book.pdf](#)

原著状态说明

原书为草稿（draft）：第 1–10 章基本完整；第 11 章（剪贴板）与第 12 章（协议扩展）的大部分小节在原著中仍是标题占位，译本如实保留，未擅自补写。

由 AutoClaw 构建

本译本完全使用 [AutoClaw](#) 构建——原书抓取、全文中英翻译、术语统一与校对、mdBook 站点生成与 PDF 导出，均由 AutoClaw 自动完成。谨此致以诚挚谢意。🙏

想体验 AutoClaw? 通过邀请链接注册：[AutoClaw 邀请链接](#)

许可与致谢

- 原文 *The Wayland Protocol* © 2021 Drew DeVault，采用 [CC BY-SA 4.0](#) 许可，原文见 <https://wayland-book.com/>。
- 中文翻译 © 2026 bitristan，同样采用 [CC BY-SA 4.0](#) 许可。

原著作者：[Drew DeVault](#)。如果你喜欢这本书，请支持原作者。

简介

Wayland 是面向类 Unix 系统的下一代显示服务器，由久负盛名的 Xorg 服务器的一批元老设计并构建，是把你的应用程序窗口显示到用户屏幕上的最佳方式。曾经使用过 X11 的读者会为 Wayland 的诸多改进而惊喜，而初次接触 Unix 图形的新手则会发现，它是一个用于构建图形应用程序与桌面环境的灵活而强大的系统。

本书将帮助你扎实地理解 Wayland 的概念、设计与实现，并为你提供构建自己的 Wayland 客户端与服务端应用所需的工具。在阅读过程中，我们将构建起 Wayland 的心智模型，并厘清其设计背后的理由。在这些篇章中，当 Wayland 那些直觉式的设计抉择变得清晰时，你应当会经历许多“原来如此！”的顿悟时刻，这也会让你欲罢不能地翻下去。欢迎来到开源图形的未来！

注意：本书仍是草稿。第 1 至 10 章大致已经完成，但日后可能还会更新。第 11 章及之后的大部分内容仍有待撰写。

TODO

- 在第 2.4 节中进一步阐述资源生命周期以及如何避免竞态条件
- 把 linux-dmabuf 的细节移到附录，并补充关于 wl_drm 与 Mesa 的说明
- 重写简介部分
- 为交互式移动添加示例代码，以演示 serial 的用法
- 准备 PDF 与 EPUB 版本

关于本书

本作品采用 Creative Commons Attribution-ShareAlike 4.0 International License 授权。源码[可在此处获取](#)。



关于作者

用 Drew 的亲密合作者 Preston Carpenter 的话来说：

Drew DeVault 在 Wayland 世界的起点是构建 sway——流行平铺式窗口管理器 i3 的一个克隆。如今无论以何种标准衡量——用户规模、提交数量还是影响力——它都是最流

行的平铺式 Wayland 合成器。在 sway 取得成功之后，Drew 通过启动 wlroots 回馈了 Wayland 社区：一套不带成见、可组合的模块，用于构建 Wayland 合成器。如今它是数十个独立合成器的基石，而 Drew 也是 Wayland 领域最顶尖的专家之一。

总体设计

你的计算机配有输入与输出设备，它们分别负责从你那里接收信息、以及把信息显示给你。这些输入设备的形式例如：

- 键盘
- 鼠标
- 触摸板
- 触摸屏
- 数位板

你的输出设备通常表现为显示器，位于你的桌上、笔记本上或移动设备上。这些资源在你的所有应用程序之间共享，而 **Wayland 合成器** 的职责就是把输入事件分派给恰当的 **Wayland 客户端**，并把它们的窗口显示在输出设备上恰当的位置。把你这所有的应用程序窗口汇集起来、一并显示到某个输出设备上的过程称为合成 (*compositing*) —— 因此我们把执行这一工作的软件称为合成器。

实践之中

桌面生态系统中存在许多彼此独立的软件组件。有用于渲染的工具如 Mesa（以及它的各个驱动），有 Linux 的 KMS/DRM 子系统，有借助 GBM 进行的缓冲区分配，有用户态的 libdrm 库，有 libinput 与 evdev，还有更多更多。别担心——理解 Wayland 并不需要你精通其中大多数系统，而且无论如何，它们大体上超出了本书的范围。事实上，Wayland 协议相当保守而抽象，一个基于 Wayland 的桌面即便完全不牵涉这些软件，也能轻松地被构建起来，并运行大多数应用程序。话虽如此，对这些部件是什么、如何工作有一个表层的了解仍是有益的。让我们从最底层开始，一路向上。

硬件

一台典型的计算机配备了几件重要的硬件。在机箱之外，有你的显示器、键盘、鼠标，也许还有些音箱和一个可爱的 USB 暖杯垫。而在机箱内部，则有若干组件负责与这些设备对接。例如，你的键盘和鼠标大概插在 USB 端口上，而你的系统为此配有专门的 USB 控制器。你的显示器则插在 GPU 上。

这些系统各有自己的职责与状态。例如，你的 GPU 拥有以内存形式存在的状态，用来存放像素缓冲区；它还有诸如把这些缓冲区*扫描输出 (scan out) *到你的显示器这样

的任务。你的 GPU 还提供了一个处理器，它被专门调校得擅长高度并行的任务（比如为一台 1080p 显示器上的 2,073,600 个像素各自算出正确的颜色），而做别的一切都很糟糕。USB 控制器则承担着令人望而生畏的职责：实现那份出了名枯燥的 USB 规范，以便从你的键盘接收输入事件，或者指示你的杯垫加热到一个经过精心挑选的温度——既能避免吃官司，又能用冷掉的咖啡让你抓狂。

在这个层面上，你的硬件几乎不知道你的系统上正运行着什么应用程序。硬件提供一个接口，人们可以命令它执行工作，而它就照做——不管是谁下的命令。正因如此，只允许一个组件与它对话.....

内核

这份责任落到了内核身上。内核是一头复杂的巨兽，所以我们只聚焦于与 Wayland 相关的那些部分。Linux 的职责是为你的硬件提供一层抽象，使其能够被用户态——也就是我们运行 Wayland 合成器的地方——安全地访问。对于图形而言，这被称为 **DRM**，即直接渲染管理器（*direct rendering manager*），它高效地把来自用户态的工作交给 GPU 去执行。DRM 的一个重要子系统是 **KMS**，即内核模式设置（*kernel mode setting*），用于枚举你的显示器并设置诸如所选分辨率（也就是它们的“模式”）之类的属性。输入设备则通过一个名为 **evdev** 的接口进行抽象。

大多数内核接口都通过 `/dev` 下的特殊文件暴露给用户态。就 DRM 而言，这些文件位于 `/dev/dri/` 下，通常以主节点（primary node，例如 `card0`）的形式存在，用于模式设置这类特权操作；以及以渲染节点（render node，例如 `renderD128`）的形式存在，用于渲染或视频解码这类非特权操作。就 evdev 而言，其“设备节点”是 `/dev/input/event*`。

用户态

现在，我们进入了用户态。在这里，应用程序与硬件相互隔离，必须经由内核提供的设备节点来工作。

libdrm

大多数 Linux 接口都有一个用户态的对应物，它提供一套还算友好的 C API 来操作这些设备节点。libdrm 就是这样一个库，它是 DRM 子系统的用户态部分。Wayland 合成器使用 libdrm 来执行模式设置以及其他 DRM 操作，但 Wayland 客户端通常不会直接使用它。

Mesa

Mesa 是 Linux 图形栈中最重要的部分之一。它提供的功能包括：面向 Linux 的、经过厂商优化的 OpenGL（以及 Vulkan）实现，以及 **GBM**（Generic 缓冲区 Management，通用缓冲区管理）库——一层位于 libdrm 之上、用于在 GPU 上分配缓冲区的抽象。大多数 Wayland 合成器都会经由 Mesa 同时使用 GBM 与 OpenGL，而大多数 Wayland 客户端至少会用到它的 OpenGL 或 Vulkan 实现。

libinput

正如 libdrm 抽象了 DRM 子系统，libinput 提供了 evdev 的用户态一端。它负责从内核接收来自你各种输入设备的输入事件，把它们解码成可用的形式，并转交给 Wayland 合成器。Wayland 合成器需要特殊权限才能使用 evdev 文件，这就迫使 Wayland 客户端必须经由合成器来接收输入事件——例如，这可以防止键盘记录。

(e)udev

处理内核中新设备的出现、为 /dev 中由此产生的设备节点配置权限、并把这类变更的消息发送给系统中运行的应用程序——这些是落在用户态头上的职责。大多数系统为此使用 udev（或其分支 eudev）。你的 Wayland 合成器使用 udev 来枚举输入设备与 GPU，并在有新设备出现或旧设备被拔出时收到通知。

xkbcommon

XKB 是 X keyboard 的缩写，是 Xorg 服务器最初的键盘处理子系统。若干年前，它从 Xorg 代码树中被提取出来，成为一套独立的键盘处理库，如今它与 X 已没有任何实际关系。libinput（连同 Wayland 合成器）以扫描码（scancode）的形式传递键盘事件，而扫描码的确切含义因键盘而异。把这些扫描码转换为有意义且通用的按键“符号”，正是 xkbcommon 的职责——例如把 65 转换为 XKB_KEY_Space。它还包含一个状态机，知道在按住 shift 时按下“1”会输出“! ”。

pixman

一个简单的库，客户端与合成器都会用它来高效地操作像素缓冲区、对相交矩形做数学运算，以及执行其他类似的**像素操控**（pixel manipulation）任务。

libwayland

libwayland 是最常用的 Wayland 协议实现，以 C 语言编写，处理了大量底层的线上协议工作。它还提供了一个工具，可从 Wayland 协议定义（即 XML 文件）生成高层代码。我们将在第 1.3 节以及贯穿全书中详细讨论 libwayland。

.....以及其他所有东西

到目前为止提到的每一个部件，在 Linux 桌面生态系统中都随处可见。除此之外，还有更多组件存在。许多图形应用程序完全不了解 Wayland，而是选择让 GTK+、Qt、SDL、GLFW 之类的库（以及许多其他库）去处理它。许多合成器选择用 wlroots 这类软件来抽象掉自己更多的职责，而另一些则把一切都自研实现。

目标与目标读者

我们的目标是让你在读完本书后，对 Wayland 协议及其高层用法有所理解。你应当对 Wayland 核心协议中的所有内容都有扎实的理解，并掌握评估和实现各种协议扩展所需的知识，而这些扩展对于高效地使用 Wayland 是必不可少的。本书主要以 Wayland 客户端的关注点为框架来展开对 Wayland 的介绍。不过，对于从事 Wayland 合成器开发工作的读者，本书也应当有一定的实用价值。

自由桌面生态系统十分复杂，由许多离散的部分构建而成。我们基本上不会讨论这些组成部分——你在本书中找不到关于如何在 Wayland 合成器中使用 libdrm，或者如何使用 libinput 处理 evdev 事件的信息。因此，本书并不是一份构建 Wayland 合成器的全面指南。我们也不会讨论那些对 Wayland 客户端有用的绘图技术，比如 Cairo、Pango、GTK+ 等等，因此本书也不是一份关于实际 Wayland 客户端实现的完备指南。相反，我们只关注 Wayland 本身的细节。

本书只涵盖协议和 libwayland。如果你正在编写客户端，而且已经熟悉了自己喜欢的用户界面渲染库，那么请带上你自己的像素，我们会帮你把它们显示到 Wayland 上。如果你已经理解了为合成器驱动显示器和输入设备所需的各项技术，本书将帮助你学会如何与客户端对话。

Wayland 软件包的内容

当你在自己所用的 Linux 发行版中安装“wayland”时，最终你很可能会得到 freedesktop.org 发布的 libwayland-client、libwayland-server、wayland-scanner 和 wayland.xml。它们分别可能位于 /usr/lib 和 /usr/include、/usr/bin 以及 /usr/share/wayland/。这个软件包是 Wayland 协议最流行的实现，但它并不是唯一的实现。第 3 章会详细讲解 Wayland 的这个实现；本书其余内容同样适用于任何实现。

wayland.xml

Wayland 协议由 XML 文件定义。如果你找到“wayland.xml”，并用自己喜欢的文本编辑器打开它，就会看到 Wayland “核心”协议的 XML 规范。这是一个高层协议——建立在我们将在下一章讨论的线上协议之上。本书的大部分篇幅都致力于解释这个文件。

wayland-scanner

“wayland-scanner”工具用于处理这些 XML 文件并从中生成代码。最常见的实现就是你现在正在查看的这个，它可用于从 wayland.xml 之类的 XML 文件生成 C 头文件和胶水代码。针对其他编程语言也存在别的扫描器，例如 wayland-rs（Rust）、waymonad-scanner（Haskell）等等。

libwayland

libwayland-client 和 libwayland-server 这两个库分别为连接的每一端提供了线上协议的实现。此外，它们还提供了一些常用的实用工具，比如处理 Wayland 数据结构的工具、一个简单的事件循环等等。另外，这些库中还包含一份由 wayland-scanner 生成的核心 Wayland 协议的预编译副本。

协议设计

Wayland 协议由若干抽象层次构建而成。它起始于一套基础的线上协议格式，即一个消息流，可借助事先约定好的接口进行解码。接着是更高层的规程：枚举接口、创建符合这些接口的资源，以及围绕它们交换消息——这就是 Wayland 协议及其扩展。在此之上，还有一些更宽泛的模式，它们在 Wayland 协议设计中被频繁使用。本章将覆盖以上全部内容。

让我们从底层开始，逐层向上推进。

线上协议基础

注意：如果你只是打算使用 libwayland，本章是可选的——可以直接跳到第 2.2 节。

线上协议是一个 32 位值的流，以主机的字节序编码（例如 x86 系列 CPU 上的小端序）。这些值表示以下基本类型：

int, uint:32 位有符号或无符号整数。

fixed:24.8 位有符号定点数。

object:32 位对象 ID。

new_id:32 位对象 ID，在收到时为该对象分配 ID。

除了这些基本类型，还使用以下其他类型：

string: 一个字符串，前置一个 32 位整数以指明其长度（以字节计），随后是字符串内容和一个 NUL 终止符，并以未定义数据填充至 32 位对齐。编码方式未加规定，但实践中使用 UTF-8。

array: 一块任意数据的 blob，前置一个 32 位整数以指明其长度（以字节计），随后是数组的原样内容，并以未定义数据填充至 32 位对齐。

fd: 在主传输上是一个 0 位值，但会借助 Unix 域套接字消息中的附属数据（msg_control）把文件描述符传递给另一端。

enum: 取自一组已知常量枚举的单个值（或位图），编码成一个 32 位整数。

消息

线上协议是由这些基本类型构建而成的消息流。每条消息都是一个作用于对象的事件（在服务端发往客户端的消息的情形下）或请求（客户端发往服务端）。

消息头包含两个“字”（word）。第一个字是所作用对象的 ID。第二个字是两个 16 位值；高 16 位是消息的大小（包括消息头自身），低 16 位是事件或请求的操作码（opcode）。消息参数紧随其后，其依据是双方事先约定好的消息签名。接收方查找该对象 ID 的接口，以及由其操作码所定义的事件或请求，以此确定消息的签名与性质。

要理解一条消息，客户端与服务端首先必须建立起这些对象。对象 ID 1 被预分配为 Wayland display 单例，可用于引导（bootstrap）其他对象。我们会在第 4 章讨论这一点。下一章将介绍接口是什么，以及在假设你已经协商好一个对象 ID 的前提下，请求和事件如何工作。说到这个.....

对象 ID

当一条消息带着 `new_id` 参数到来时，发送方会为它分配一个对象 ID——用于该对象的接口通过额外的参数来确立，或者针对该请求/事件事先约定好。这个对象 ID 可以用于后续消息，要么作为消息头的第一个字，要么作为 `object_id` 参数。客户端分配的 ID 范围是 `[1, 0xFEFFFFFF]`，服务端分配的 ID 范围是 `[0xFF000000, 0xFFFFFFFF]`。ID 从该边界下端开始，每次分配新对象时递增。

对象 ID 为 0 表示空对象（null object）；也就是不存在的对象，或显式地表示没有对象。

传输

迄今为止，所有已知的 Wayland 实现都工作在 Unix 域套接字之上。这么做有一个特别的原因：文件描述符消息。Unix 套接字是能够在进程之间传递文件描述符的最务实的传输方式，而这对于大数据量传输是必需的（键盘映射、像素缓冲区以及剪贴板内容是主要的用例）。理论上，也可以使用别的传输方式（例如 TCP），但总得有人想出另一种传递批量数据的办法。

为了找到要连接的 Unix 套接字，大多数实现就照 libwayland 的做法来：

1. 如果设置了 `WAYLAND_SOCKET`，就把它解释为一个文件描述符编号，连接在该描述符上已经建立，并假定父进程已经为我们配置好了连接。
2. 如果设置了 `WAYLAND_DISPLAY`，就与 `XDG_RUNTIME_DIR` 拼接，构成 Unix 套接字的路径。
3. 假定套接字名称为 `wayland-0`，并与 `XDG_RUNTIME_DIR` 拼接，构成 Unix 套接字的路径。
4. 放弃。

接口、请求与事件

Wayland 协议通过发出作用于对象的请求和事件来运作。每个对象都有一个接口，它定义了可能有哪些请求和事件，以及各自的签名。我们来看一个示例接口：wl_surface。

请求

一个表面是一块可以被显示在屏幕上的像素盒子。它是我们用来构建诸如应用程序窗口之类东西的基本单元之一。它的其中一个请求——由客户端发往服务端——是“damage”，客户端用它来表明表面的某部分已经改变、需要重绘。下面是一个在线上传输的“damage”消息的带注释示例（十六进制）：

```
00000000A    Object ID (10)
00180002    Message length (24) and request opcode (2)
00000000    X coordinate (int): 0
00000000    Y coordinate (int): 0
00000100    Width          (int): 256
00000100    Height         (int): 256
```

这是一段会话的片段——该表面早先已被分配，并被赋予 ID 10。当服务端收到这条消息时，它查找 ID 为 10 的对象，发现它是一个 wl_surface 实例。知道这一点后，它查找操作码为 2 的请求的签名。于是它就知道要预期四个整数作为参数，便可以对消息进行解码，并在内部派发处理。

事件

服务端也可以向客户端发回消息——事件。服务端可以针对 wl_surface 发送的其中一个事件是“enter”，它在某个特定 output 上显示该表面时发送（例如，客户端可以据此调整它在 HiDPI 显示器上的缩放系数作为响应）。下面是这样一个消息的示例：

```
00000000A    Object ID (10)
000C0000    Message length (12) and event opcode (0)
00000005    Output (object ID): 5
```

这条消息通过 ID 引用了另一个对象：即该表面正在其上显示的那个 wl_output 对象。客户端收到它之后，会和刚才的服务端跳同一支舞。它查找对象 10，将其与

`wl_surface` 接口关联起来，并查找操作码 0 所对应事件的签名。它相应地解码消息的其余部分，同时也查找 ID 为 5 的 `wl_output`，然后在内部派发处理。

接口

那些定义请求与事件列表、各自关联的操作码，以及可用于解码消息的签名的接口，都是事先约定好的。我相信你迫不及待地想知道是怎么做到的——翻到下一页，悬念就此揭晓。

高层协议

在第 1.3 节中，我提到 `wayland.xml` 很可能已随你系统上的 Wayland 软件包一起安装了。现在就用你最喜欢的文本编辑器找到并打开那个文件。正是通过这个文件，以及其他类似的文件，我们定义了某个 Wayland 客户端或服务端所支持的接口。

每个接口都在这个文件中定义，连同它的请求和事件，以及各自对应的签名。为此我们使用 XML——这个人人最爱的文件格式。来看一看我们在上一章讨论过的 `wl_surface` 的那些示例。下面是一个样例：

```
<interface name="wl_surface" version="4">
  <request name="damage">
    <arg name="x" type="int" />
    <arg name="y" type="int" />
    <arg name="width" type="int" />
    <arg name="height" type="int" />
  </request>

  <event name="enter">
    <arg name="output" type="object" interface="wl_output" />
  </event>
</interface>
```

注意：为了简洁，我裁剪了这段片段，但如果你面前有 `wayland.xml` 文件，请找出这个接口并亲自检视它——其中还附有更多文档，解释了每个请求和事件的目的与确切语义。

在处理这个 XML 文件时，我们按每个请求和事件出现的顺序为其分配一个操作码，从零开始计数，并各自独立递增。结合参数列表，当请求或事件经由线上协议到来时，你就能对其进行解码；并且根据 XML 文件中附带的文档，你可以决定如何为你的软件编程以作出相应的行为。这通常以代码生成的形式实现——我们会在第 3 章讨论 `libwayland` 是如何做到这一点的。

从第 4 章开始，本书剩余的大部分内容都将用来讲解这个文件，以及一些补充性的协议扩展。

协议设计模式

有一些关键概念已被应用于大多数 Wayland 协议的设计之中，我们应当在此简要介绍。这些模式贯穿于整个高层 Wayland 协议及协议扩展（嗯，至少在 Wayland 协议里是如此¹）。如果你发现自己正在编写自己的协议扩展，明智的做法是同样应用这些模式。

原子性

Wayland 协议设计模式中最重要的一项是原子性。Wayland 的一个既定目标是“每一帧都完美无缺”。为此，大多数接口都允许你以事务方式对其进行更新——先用若干请求构建出其状态的全新表示，然后一次性将它们提交。例如，可以在 `wl_surface` 上配置的属性就有如下几项：

- 一个已附加的像素缓冲区
- 一块需要重绘的损伤区域
- 一个出于优化目的而被定义为不透明的区域
- 一个可接受输入事件的区域
- 一种变换，例如旋转 90 度
- 一个用于 HiDPI 的缓冲区缩放值

该接口为配置上述每一项属性都提供了单独的请求，但这些修改会被应用到 *pending*（待定）状态上。只有当 **提交** 请求发出后，待定状态才会被合并进 *current*（当前）状态，从而让你能够在单帧之内以原子方式更新所有这些属性。再结合其他几个关键的设计决策，这使得 Wayland 合成器能够在每一帧都完美地渲染一切——没有撕裂，也没有只更新到一半的窗口，每个像素都恰在其位，每个位置都有其像素。

资源生命周期

另一个重要的设计模式，是避免服务端或客户端发送与已失效对象相关的请求或事件。为此，那些定义了寿命有限的资源的接口，往往会包含一些请求和事件，客户端或服务端可借此声明自己不再会为该对象发送请求或事件。只有当双方都——异步地——就这一点达成一致之后，它们才会销毁各自为该对象分配的资源。

Wayland 是一个完全异步的协议。消息保证按发送顺序到达，但这只是相对于单个发送方而言。例如，当客户端决定销毁它的键盘设备时，服务端可能已经排入了若干输入事

件。客户端必须正确处理这些针对它已不再需要的对象的事件，直到服务端跟上进度为止。同样地，倘若客户端在销毁某个对象之前已为其排入了若干请求，那么这些请求就必须以正确的顺序发送，使得在客户端同意该对象已被销毁之后，它不再被使用。

版本管理

Wayland 协议中使用的版本管理模型有两种：不稳定和稳定。这两种模型都只允许向后兼容的变更，但当协议从不稳定过渡到稳定时，允许进行最后一次破坏性变更。这为协议提供了一个孵化期，在此期间我们可以在实践中测试它们，然后将所得的经验心得应用到最后一轮大规模破坏性变更之中，从而打造出一个应当能经受时间考验的协议²。

要进行向后兼容的变更，你只能在接口末尾添加新的事件或请求，或在枚举末尾添加新成员。此外，每个实现都必须自我约束，只使用连接另一端所支持的消息。我们将在第 5 章讨论如何确定连接的每一方分别在使用各接口的哪个版本。

-
1. 除了那一个接口。你看，至少我们尝试过了，对吧？↩
 2. 请注意，在撰写本文时，许多有用的协议仍处于不稳定状态。它们可能有点粗糙，但仍被广泛使用，这正是向后兼容性重要的原因。将协议从不稳定升级为稳定时，采用的升级方式允许软件同时支持不稳定和稳定的协议，从而实现更平滑的过渡。↩

libwayland 实现

我们在第 1.3 节中简要介绍过 libwayland——它是最流行的 Wayland 实现。本书的大部分内容适用于任何实现，但接下来的两章里，我们将带你熟悉这一个实现。

Wayland 软件包为 wayland-client 和 wayland-server 提供了 pkg-config 规范——如何与它们链接，请查阅你所用的构建系统的文档。自然，大多数应用程序只会链接其中之一。该库包含一些简单的原语（比如链表），以及一份预编译的 `wayland.xml`——即 Wayland 核心协议。

我们将从介绍这些原语开始。

wayland-util 原语

客户端和服务端库共用的头文件是 `wayland-util.h`，其中定义了若干结构体、实用函数和宏，为 Wayland 应用程序奠定了一批原语。其中包括：

- 用于在生成代码中对 Wayland 协议消息进行序列化与反序列化的结构
- 链表 (`wl_list`) 实现
- 数组 (`wl_array`) 实现（与对应的 Wayland 原语相挂钩）
- 在 Wayland 标量（例如定点数）与 C 类型之间进行转换的工具
- 用于从 `libwayland` 内部向外传递信息的调试日志设施

这个头文件本身就带有许多注释，文档写得相当不错——你应该亲自通读一遍。在接下来的几页中，我们将详细讲解如何运用这些原语。

wayland-scanner

Wayland 软件包附带一个二进制文件：wayland-scanner。这个工具用于从第 2.3 节讨论过的 Wayland 协议 XML 文件生成 C 头文件和胶水代码。在“wayland”软件包的构建过程中，这个工具被用来为核心协议 wayland.xml 预先生成头文件和胶水代码。这些头文件会成为 wayland-client-protocol.h 和 wayland-server-protocol.h——不过你通常会包含 wayland-client.h 和 wayland-server.h，而不是直接使用它们。

这个工具的用法相当简单（wayland-scanner -h 有概要说明），可以归纳如下。生成客户端头文件：

```
wayland-scanner client-header < protocol.xml > protocol.h
```

生成服务端头文件：

```
wayland-scanner server-header < protocol.xml > protocol.h
```

生成胶水代码：

```
wayland-scanner private-code < protocol.xml > protocol.c
```

不同的构建系统配置自定义命令的方式各不相同——请查阅你的构建系统的文档。一般来说，你会在构建时运行 wayland-scanner，然后把你的应用程序与生成的胶水代码一起编译和链接。

现在就随便拿一个 Wayland 协议来试试吧，如果你手头正好有的话（例如，wayland.xml 很可能就在 /usr/share/wayland 里）。打开生成的胶水代码和头文件，在阅读后续章节时随手对照，以便理解 libwayland 提供的原语在生成代码中是如何被实际运用的。

代理与资源

对象（object）是客户端和服务端双方都知晓的实体，它拥有一些状态，对这些状态的更改通过线上协议协商进行。在客户端一侧，libwayland 通过 `wl_proxy` 接口来引用这些对象。它们是抽象对象在 C 中的一个具体、友好的“代理”，并提供一些函数供客户端间接地用来将请求序列化为线上格式。如果你查看 `wayland-client-core.h` 文件，会发现若干为此目的而设的底层函数。一般来说，你不会直接使用它们。

在服务端，对象通过 `wl_resource` 来抽象，二者相当相似，但 `wl_resource` 多了一层复杂度——服务端必须跟踪哪个对象属于哪个客户端。每个 `wl_resource` 都为单个客户端所拥有。除此之外，两者的接口大同小异，`wl_resource` 为序列化事件并发送给相关客户端提供了底层抽象。在服务端上，你直接使用 `wl_resource` 的场合要比在客户端上直接接触 `wl_proxy` 接口更多。这类用法的一个例子是：获取拥有某个资源的 `wl_client` 的引用（当你在脱离上下文的情况下操作该资源时），或者当客户端尝试执行无效操作时向其发送协议错误。

再往上一层是另一组更高层的接口，大多数 Wayland 客户端和服务端正是通过与它们交互来完成其大部分任务的。我们将在下一节中介绍它们。

接口与监听器

最后，我们终于抵达 libwayland 抽象的顶峰：接口与监听器。前面几章讨论过的那些概念——wl_proxy 和 wl_resource，以及各种原语——是存在于 libwayland 中的单一实现，它们的存在正是为了给这一层提供支持。当你把一个 XML 文件交给 wayland-scanner 处理时，它会生成接口和监听器，以及它们与底层线上协议接口之间的胶水代码，全都对应于高层协议中的每个接口。

回想一下，Wayland 连接上的每个参与者既可以接收也可以发送消息。客户端监听事件并发送请求，而服务端监听请求并发送事件。每一方都通过一个名字起得很贴切的 wl_listener 来监听对方的消息。下面是这个接口的一个例子：

```
struct wl_surface_listener {
    /** surface enters an output */
    void (*enter)(void *data,
                  struct wl_surface *wl_surface,
                  struct wl_output *output);

    /** surface leaves an output */
    void (*leave)(void *data,
                  struct wl_surface *wl_surface,
                  struct wl_output *output);
};
```

这是 wl_surface 的一个客户端监听器。wayland-scanner 用来生成它的 XML 是：

```
<interface name="wl_surface" version="4">
  <event name="enter">
    <arg name="output"
          type="object"
          interface="wl_output"/>
  </event>

  <event name="leave">
    <arg name="output"
          type="object"
          interface="wl_output"/>
  </event>
  <!-- additional details omitted for brevity -->
</interface>
```

这些事件如何变成一个监听器接口，应该相当清楚了。每个函数指针都接收一份任意的用户数据、一个事件所指向的资源的引用，以及该事件的参数。我们可以像这样把一个监听器绑定到 wl_surface 上：

```

static void wl_surface_enter(void *data,
                             struct wl_surface *wl_surface, struct wl_output *output) {
    // ...
}

static void wl_surface_leave(void *data,
                             struct wl_surface *wl_surface, struct wl_output *output) {
    // ...
}

static const struct wl_surface_listener surface_listener = {
    .enter = wl_surface_enter,
    .leave = wl_surface_leave,
};

// ...

struct wl_surface *surf;
wl_surface_add_listener(surf, &surface_listener, NULL);

```

wl_surface 接口还定义了客户端可以针对该表面发起的一些请求：

```

<interface name="wl_surface" version="4">
  <request name="attach">
    <arg name="buffer"
        type="object"
        interface="wl_buffer"
        allow-null="true"/>
    <arg name="x" type="int"/>
    <arg name="y" type="int"/>
  </request>
  <!-- additional details omitted for brevity -->
</interface>

```

wayland-scanner 会生成如下函数原型，以及负责将这条消息序列化的胶水代码。

```

void wl_surface_attach(struct wl_surface *wl_surface,
                      struct wl_buffer *buffer, int32_t x, int32_t y);

```

接口与监听器的服务端代码与之完全相同，只是方向相反——它为请求生成监听器，为事件生成胶水代码。当 libwayland 收到一条消息时，它会查找对象 ID 及其接口，再用它来解码消息的其余部分。然后它在这个对象上寻找监听器，并带上消息的参数来调用你的函数。

就是这么回事！我们经过了好几层抽象才走到这里，但你现在应该已经理解一个事件如何从你的服务端代码出发、变成线上的一条消息、被客户端理解、并最终分发到你的客户端代码。不过，还有一个问题没有回答。所有这一切都预设你已经拿到了 Wayland 对象的引用。那么，这些引用是从哪里来的呢？

Wayland display

到目前为止，在我们讲解 Wayland 协议如何在客户端与服务端之间对对象进行共同所有权管理时，一直遗漏了一个关键细节：这些对象最初究竟是如何被创建的。Wayland display，即 `wl_display`，隐式地存在于每一条 Wayland 连接之上。它拥有如下接口：

```
<interface name="wl_display" version="1">
  <request name="sync">
    <arg name="callback" type="new_id" interface="wl_callback"
      summary="callback object for the sync request"/>
  </request>

  <request name="get_registry">
    <arg name="registry" type="new_id" interface="wl_registry"
      summary="global registry object"/>
  </request>

  <event name="error">
    <arg name="object_id" type="object" summary="object where the
error occurred"/>
    <arg name="code" type="uint" summary="error code"/>
    <arg name="message" type="string" summary="error description"/>
  </event>

  <enum name="error">
    <entry name="invalid_object" value="0" />
    <entry name="invalid_method" value="1" />
    <entry name="no_memory" value="2" />
    <entry name="implementation" value="3" />
  </enum>

  <event name="delete_id">
    <arg name="id" type="uint" summary="deleted object ID"/>
  </event>
</interface>
```

对一般 Wayland 用户而言，其中最有趣的是 `get_registry`，我们将在下一章详细讨论它。简而言之，注册表用于分配其他对象。接口的其余部分则用于对连接做日常维护，除非你打算编写自己的 `libwayland` 替代品，否则通常并不重要。

相反，本章将聚焦于 `libwayland` 与 `wl_display` 对象相关联的一组函数，它们用于建立并维护你的 Wayland 连接。这些函数用于操作 `libwayland` 的内部状态，而与线上协议的请求和事件没有直接关系。

我们将从这些函数中最重要的一项开始：建立 display。对客户端而言，这会涵盖连接服务端的实际过程；对服务端而言，则是配置一个供客户端连接的 display 的过程。

创建 display

打开你的文本编辑器——是时候写下我们的第一行代码了。

面向 Wayland 客户端

连接到一个 Wayland 服务端，并创建一个 `wl_display` 来管理该连接的状态，是相当的容易的：

```
#include <stdio.h>
#include <wayland-client.h>

int
main(int argc, char *argv[])
{
    struct wl_display *display = wl_display_connect(NULL);
    if (!display) {
        fprintf(stderr, "Failed to connect to Wayland display.\n");
        return 1;
    }
    fprintf(stderr, "Connection established!\n");

    wl_display_disconnect(display);
    return 0;
}
```

让我们编译并运行这个程序。假设你阅读本书时正使用某个 Wayland 合成器，结果应当如下所示：

```
$ cc -o client client.c -lwayland-client
$ ./client
Connection established!
```

`wl_display_connect` 是客户端建立 Wayland 连接最常见的方式。其签名为：

```
struct wl_display *wl_display_connect(const char *name);
```

“name” 参数是 Wayland display 的名称，通常是 “wayland-0”。你可以在我们的测试客户端中把那个 `NULL` 换成它，亲自试一试——它很可能会奏效。它对应的是

`$XDG_RUNTIME_DIR` 中某个 Unix 套接字的名称。不过更推荐使用 `NULL`，此时 `libwayland` 会：

1. 如果设置了 `$WAYLAND_DISPLAY`，则尝试连接 `$XDG_RUNTIME_DIR/$WAYLAND_DISPLAY`
2. 否则，尝试连接 `$XDG_RUNTIME_DIR/wayland-0`
3. 否则，失败 :(

这样一来，用户就可以通过把 `$WAYLAND_DISPLAY` 设置为想要的 display，来指定他们想让客户端运行在哪个 Wayland display 上。如果你有更复杂的需求，也可以自行建立连接，并从某个文件描述符创建 Wayland display：

```
struct wl_display *wl_display_connect_to_fd(int fd);
```

无论你以何种方式创建 display，也都可以通过 `wl_display_get_fd` 获取该 `wl_display` 正在使用的文件描述符。

```
int wl_display_get_fd(struct wl_display *display);
```

面向 Wayland 服务端

对服务端而言，过程也相当简单。display 的创建与套接字的绑定是分开进行的，以便你有时间在任何客户端能够连接到它之前对 display 进行配置。下面是另一个最小示例程序：

```

#include <stdio.h>
#include <wayland-server.h>

int
main(int argc, char *argv[])
{
    struct wl_display *display = wl_display_create();
    if (!display) {
        fprintf(stderr, "Unable to create Wayland display.\n");
        return 1;
    }

    const char *socket = wl_display_add_socket_auto(display);
    if (!socket) {
        fprintf(stderr, "Unable to add socket to Wayland display.\n");
        return 1;
    }

    fprintf(stderr, "Running Wayland display on %s\n", socket);
    wl_display_run(display);

    wl_display_destroy(display);
    return 0;
}

```

让我们也把它编译并运行一下：

```

$ cc -o server server.c -lwayland-server
$ ./server &
Running Wayland display on wayland-1
$ WAYLAND_DISPLAY=wayland-1 ./client
Connection established!

```

使用 `wl_display_add_socket_auto` 会让 libwayland 自动决定 display 的名称，其默认为 `wayland-0`，或者 `wayland-$n`，具体取决于 `$XDG_RUNTIME_DIR` 中是否已有其他 Wayland 合成器的套接字。不过，和客户端一样，你在配置 display 时还有一些其他选择：

```

int wl_display_add_socket(struct wl_display *display, const char
*name);

int wl_display_add_socket_fd(struct wl_display *display, int sock_fd);

```

添加套接字之后，调用 `wl_display_run` 会运行 libwayland 内部的事件循环，并阻塞直到 `wl_display_terminate` 被调用。这个事件循环是什么？翻到下一页便知！

引入事件循环

libwayland 自带一套事件循环实现，供 Wayland 服务端使用，但维护者已承认这属于一次设计上的越界。对客户端而言则没有这样的等价物。尽管如此，即便超出了本书的讨论范围，Wayland 服务端的事件循环也足够实用。

Wayland 服务端事件循环

libwayland-server 创建的每个 `wl_display` 都有一个对应的 `wl_event_loop`，你可以通过 `wl_display_get_event_loop` 获取对它的引用。如果你在编写一个新的 Wayland 合成器，你很可能会想把它作为你唯一的事件循环。你可以用 `wl_event_loop_add_fd` 向它添加文件描述符，用 `wl_event_loop_add_timer` 添加定时器。它还通过 `wl_event_loop_add_signal` 处理信号，用起来相当方便。

把事件循环按你的喜好配置好，以监视你的合成器必须响应的所有事件之后，你就可以通过调用 `wl_display_run` 一次性处理事件并派发 Wayland 客户端，它会运行事件循环并阻塞，直到 display 终止（经由 `wl_display_terminate`）。大多数从一开始就以 Wayland 为本（而非从 X11 移植）构建的 Wayland 合成器都采用这种做法。

不过，你也可以亲自掌舵，把 Wayland display 纳入你自己的事件循环。`wl_display` 在内部使用该事件循环来处理客户端，你可以选择自行监视 Wayland 事件循环并按需派发它，也可以完全无视它、手动处理客户端更新。如果你希望让 Wayland 事件循环自己照顾自己，把它当作你自己事件循环的从属，你可以用 `wl_event_loop_get_fd` 获得一个可 `poll` 的文件描述符，然后在该文件描述符上出现活动时调用 `wl_event_loop_dispatch` 来处理事件。当你存在需要写入客户端的数据时，还需要调用 `wl_display_flush_clients`。

Wayland 客户端事件循环

另一方面，libwayland-client 没有自己的事件循环。不过，由于通常只有一个文件描述符，没有它反而更容易管理。如果 Wayland 事件是你的程序唯一需要处理的那类事件，那么下面这个简单循环就足够了：

```
while (wl_display_dispatch(display) != -1) {  
    /* This space deliberately left blank */  
}
```

然而，如果你的应用程序更为复杂，你可以按自己乐意的方式构建自己的事件循环，并用 `wl_display_get_fd` 获取 Wayland display 的文件描述符。在发生 `POLLIN` 事件时，调用 `wl_display_dispatch` 来处理传入事件。要冲刷传出的请求，请调用 `wl_display_flush`。

就快到了！

此时，你已具备搭建 Wayland display、处理事件与请求所需的全部背景知识。剩下的唯一一步，就是分配一些对象，用来与连接的另一端交谈。为此，我们使用注册表。到下一章结束时，我们将拥有第一个真正有用的 Wayland 客户端！

全局对象与注册表

如果你还记得第 2.1 节的内容，每个请求和事件都与一个对象 ID 相关联，但到目前为止我们还没有讨论过对象是如何创建的。当我们收到一条 Wayland 消息时，我们必须知道该对象 ID 代表的是哪个接口，才能对其进行解码。我们还必须以某种方式协商可用对象、新对象的创建，以及为它们分配 ID。在 Wayland 中，我们一次性解决了这两个问题——当我们绑定一个对象 ID 时，我们就此约定在所有未来的消息中用于该对象的接口，并在本地状态中保存一份对象 ID 到接口的映射。

为了给这一切做引导，服务端提供了一份全局对象的列表。这些全局对象本身往往就以其自身的价值提供信息和功能，但最常见的情况是，它们被用来代理其他对象，以满足各种用途，例如创建应用程序窗口。这些全局对象本身也有自己的对象 ID 和接口，我们必须以某种方式分配并约定它们。

既然现在你脑海中无疑已经浮现出先有鸡还是先有蛋的问题，那我就来揭示这个秘密诀窍：当你建立连接时，对象 ID 1 已经被隐式地分配给了 `wl_display` 接口。回顾一下该接口，请注意其中的 `wl_display::get_registry` 请求：

```
<interface name="wl_display" version="1">
  <request name="sync">
    <arg name="callback" type="new_id" interface="wl_callback" />
  </request>

  <request name="get_registry">
    <arg name="registry" type="new_id" interface="wl_registry" />
  </request>

  <!-- ... -->
</interface>
```

`wl_display::get_registry` 请求可用于将一个对象 ID 绑定到 `wl_registry` 接口，它是 `wayland.xml` 中接着出现的下一个接口。鉴于 `wl_display` 始终拥有对象 ID 1，下面这条线上消息应该就说得通了（采用大端序）：

```
C->S      00000001 000C0001 00000002      .... ....
```

当我们把它拆解开来时，第一个数字是对象 ID。第二个数字的最高 16 位是消息的总长度（以字节为单位），最低位则是请求的操作码。其余的字（这里只有一个）是参数。简而言之，这在对象 ID 1 (`wl_display`) 上调用请求 1（从 0 开始计数），该请求接受一个参数：一个新生成的、供新对象使用的 ID。请注意 XML 文档中，这个新 ID 已被预先定义为受 `wl_registry` 接口支配：

```
<interface name="wl_registry" version="1">
  <request name="bind">
    <arg name="name" type="uint" />
    <arg name="id" type="new_id" />
  </request>

  <event name="global">
    <arg name="name" type="uint" />
    <arg name="interface" type="string" />
    <arg name="version" type="uint" />
  </event>

  <event name="global_remove">
    <arg name="name" type="uint" />
  </event>
</interface>
```

我们将在接下来的章节中讨论的正是这个接口。

绑定全局对象

创建注册表对象后，服务端会为服务端上可用的每一个全局对象发出 `global` 事件。随后你就可以绑定到你所需的全局对象。

绑定是指取出一个已知对象并为其分配 ID 的过程。一旦客户端像这样绑定到注册表，服务端就会多次发出 `global` 事件，以通告它支持哪些接口。每个这样的全局对象都会被分配一个唯一的 `name`，形式为一个无符号整数。`interface` 字符串对应的是协议中所见的接口名称：上面 XML 中的 `wl_display` 就是这样一个名称的示例。版本号也在这里定义——关于接口版本的更多信息，参见附录 C。

要绑定到其中任何一个接口，我们使用 `bind` 请求，它的工作方式与我们绑定到 `wl_registry` 的那个神奇过程类似。例如，考虑下面这段线上协议交互：

```
C->S      00000001 000C0001 00000002      .... .... ....

S->C      00000002 001C0000 00000001 00000007      .... .... .... ....
          776C5f73 686d0000 00000001              wl_s hm.. ....
          [...]

C->S      00000002 00100000 00000001 00000003      .... .... .... ....
```

第一条消息与我们已经剖析过的那条完全相同。第二条是来自服务端的事件：对象 2（客户端在第一条消息中把 `wl_registry` 分配给了它）操作码 0（“`global`”），参数为 1、“`wl_shm`”和 1——分别是这个全局对象的 `name`、接口和 `version`。客户端通过在对象 ID 2（`wl_registry::bind`）上调用操作码 0 来响应，并把对象 ID 3 分配给全局对象 `name 1`——即绑定到 `wl_shm` 全局对象。该对象未来的事件和请求由 `wl_shm` 协议定义，你可以在 `wayland.xml` 中找到它。

一旦创建了这个对象，你就可以利用它的接口来完成各种任务——就 `wl_shm` 而言，是在客户端与服务端之间管理共享内存。本书剩余的大部分内容都致力于讲解这些全局对象中每一个的用法。

有了这些信息，我们就可以编写第一个有用的 Wayland 客户端了：一个只是简单地打印出服务端上所有可用全局对象的程序。

```

#include <stdint.h>
#include <stdio.h>
#include <wayland-client.h>

static void
registry_handle_global(void *data, struct wl_registry *registry,
                      uint32_t name, const char *interface, uint32_t
version)
{
    printf("interface: '%s', version: %d, name: %d\n",
          interface, version, name);
}

static void
registry_handle_global_remove(void *data, struct wl_registry
*registry,
                      uint32_t name)
{
    // This space deliberately left blank
}

static const struct wl_registry_listener
registry_listener = {
    .global = registry_handle_global,
    .global_remove = registry_handle_global_remove,
};

int
main(int argc, char *argv[])
{
    struct wl_display *display = wl_display_connect(NULL);
    struct wl_registry *registry =
wl_display_get_registry(display);
    wl_registry_add_listener(registry, &registry_listener, NULL);
    wl_display_roundtrip(display);
    return 0;
}

```

解读这个程序时，请随意参考前面的章节。我们连接到 display（第 4.1 节），获取注册表（本章），为它添加一个监听器（第 3.4 节），然后进行往返（round-trip），通过打印此合成器上可用的全局对象来处理 global 事件。你自己动手试试：

```
$ cc -o globals -lwayland-client globals.c
```

注意：本章是我们最后一次以十六进制形式展示线上协议转储，而且这大概也是你今后最后一次见到它们了。跟踪你的 Wayland 客户端或服务端的一个更好的方法，是在运行程序之前把环境中的 WAYLAND_DEBUG 变量设置为 1。现在就用“globals”程序试一下吧！

注册全局对象

用 libwayland-server 注册全局对象的方式略有不同。当你用 wayland-scanner 生成 “server-code”（服务端代码）时，它会创建接口（类似于监听器）以及用于发送事件的胶水代码。第一项任务是注册全局对象，并提供一个函数，以便在全局对象被绑定时装配起一个资源¹。用代码来表示，结果大概长这样：

```
static void
wl_output_handle_bind(struct wl_client *client, void *data,
    uint32_t version, uint32_t id)
{
    struct my_state *state = data;
    // TODO
}

int
main(int argc, char *argv[])
{
    struct wl_display *display = wl_display_create();
    struct my_state state = { ... };
    // ...
    wl_global_create(wl_display, &wl_output_interface,
        1, &state, wl_output_handle_bind);
    // ...
}
```

如果你把这段代码，比如，补进第 4.1 节的服务端示例中，你就会让一个 wl_output 全局对象对我们上次编写的 “globals” 程序可见²。不过，任何绑定到这个全局对象的尝试都会撞上我们的 TODO。为了填补它，我们还需要为 wl_output 接口提供一个实现。

```

static void
wl_output_handle_resource_destroy(struct wl_resource *resource)
{
    struct my_output *client_output =
wl_resource_get_user_data(resource);

    // TODO: Clean up resource

    remove_to_list(client_output->state->client_outputs,
client_output);
}

static void
wl_output_handle_release(struct wl_client *client, struct wl_resource
*resource)
{
    wl_resource_destroy(resource);
}

static const struct wl_output_interface
wl_output_implementation = {
    .release = wl_output_handle_release,
};

static void
wl_output_handle_bind(struct wl_client *client, void *data,
    uint32_t version, uint32_t id)
{
    struct my_state *state = data;

    struct my_output *client_output = calloc(1, sizeof(struct
client_output));

    struct wl_resource *resource = wl_resource_create(
        client, &wl_output_interface, wl_output_interface.version,
id);

    wl_resource_set_implementation(resource,
&wl_output_implementation,
        client_output, wl_output_handle_resource_destroy);

    client_output->resource = resource;
    client_output->state = state;

    // TODO: Send geometry event, et al

    add_to_list(state->client_outputs, client_output);
}

```

要消化的东西很多，所以我们一小块一小块地来解释。在最底部，我们扩展了“bind”处理函数，创建一个 `wl_resource` 来跟踪这个对象在服务端一侧的状态（使用客户端分配的那个 ID）。在此过程中，我们把指向接口实现的指针——

`wl_output_implementation`，本文件中的一个常量静态结构体——传给 `wl_resource_create`。该类型 (`struct wl_output_interface`) 由 `wayland-scanner` 生成，为此接口所支持的每一个请求包含一个函数指针。我们也借此机会分配一个小容器，用来存储 `libwayland` 不替我们处理的任何额外状态，其具体性质因协议而异。

注意： 这里有两个不同的东西共用了同一个名称：`struct wl_output_interface` 是接口的一个实例，而 `wl_output_interface` 是由 `wayland-scanner` 生成的一个全局常量变量，其中包含与该实现相关的元数据（例如上面例子中使用的 `version`）。

我们的 `wl_output_handle_release` 函数会在客户端发送 `release` 请求时被调用，表明它不再需要这个资源——于是我们销毁它。这又会触发 `wl_output_handle_resource_destroy` 函数，稍后我们会扩展它，以释放我们先前为其分配的任何状态。此函数也作为析构器被传入 `wl_resource_create`，如果客户端没有显式发送 `release` 请求就终止了，它就会被调用。

我们代码中另一个剩下的“TODO”是发送“name”事件，以及其他几个事件。如果我们查阅一下 `wayland.xml`，就会在接口上看到这个事件：

```
<event name="geometry">
  <description summary="properties of the output">
    The geometry event describes geometric properties of the output.
    The event is sent when binding to the output object and whenever
    any of the properties change.

    The physical size can be set to zero if it doesn't make sense for this
    output (e.g. for projectors or virtual outputs).
  </description>
  <arg name="x" type="int" />
  <arg name="y" type="int" />
  <arg name="physical_width" type="int" />
  <arg name="physical_height" type="int" />
  <arg name="subpixel" type="int" enum="subpixel" />
  <arg name="make" type="string" />
  <arg name="model" type="string" />
  <arg name="transform" type="int" enum="transform" />
</event>
```

看来在 `output` 被绑定时发送这个事件是我们的职责。要加上它很容易：

```

static void
wl_output_handle_bind(struct wl_client *client, void *data,
    uint32_t version, uint32_t id)
{
    struct my_state *state = data;

    struct my_output *client_output = calloc(1, sizeof(struct
client_output));

    struct wl_resource *resource = wl_resource_create(
        client, &wl_output_implementation,
wl_output_interface.version, id);

    wl_resource_set_implementation(resource, wl_output_implementation,
        client_output, wl_output_handle_resource_destroy);

    client_output->resource = resource;
    client_output->state = state;

    wl_output_send_geometry(resource, 0, 0, 1920, 1080,
        WL_OUTPUT_SUBPIXEL_UNKNOWN, "Foobar, Inc",
        "Fancy Monitor 9001 4K HD 120 FPS Noscope",
        WL_OUTPUT_TRANSFORM_NORMAL);

    add_to_list(state->client_outputs, client_output);
}

```

注意：这里展示 `wl_output::geometry` 只是出于说明目的，但在实践中它的使用有一些特殊的注意事项。在你的客户端或服务端中实现这个事件之前，请先审阅协议 XML。

-
1. 资源代表每个客户端对某个对象的实例在服务端一侧的状态。 [↩](#)
 2. 如果你对更健壮的东西感兴趣，Weston 项目提供了一个我们“globals”程序的略为精巧的版本，叫做 `weston-info`。 [↩](#)

缓冲区与表面

显然，这个系统的全部意义就在于向用户显示信息，并接收他们的反馈以进行进一步处理。本章中，我们将探讨其中的第一项任务：在屏幕上显示像素。

为实现这一目的，有两个基础原语：缓冲区和表面，分别由 `wl_buffer` 和 `wl_surface` 接口管理。缓冲区充当某种底层像素存储的不透明容器，客户端可以通过多种方式提供它——最常见的是共享内存缓冲区和 GPU 句柄。

使用 wl_compositor

人们说，命名是计算机科学中最困难的问题之一，而此刻我们正手握证据站在这里。wl_compositor 全局对象就是 Wayland 合成器的、呃，合成器。通过这个接口，你可以把你的窗口发送给服务端进行呈现，以便与同时显示在旁边的其他窗口一起被合成。合成器有两项职责：创建表面和区域。

套用规范的说法，一个 Wayland 表面 拥有一个矩形区域，它可以显示在零个或多个输出上、呈现缓冲区、接收用户输入，并定义一个本地坐标系。这些内容我们稍后都会逐一详细拆解，但让我们先从基础开始：获取一个表面并向其附加一个缓冲区。要获取一个表面，我们首先绑定 wl_compositor 全局对象。把第 5.1 节的示例加以扩展，就得到如下代码：

```
struct our_state {
    // ...
    struct wl_compositor *compositor;
    // ...
};

static void
registry_handle_global(void *data, struct wl_registry *wl_registry,
                       uint32_t name, const char *interface, uint32_t
version)
{
    struct our_state *state = data;
    if (strcmp(interface, wl_compositor_interface.name) == 0) {
        state->compositor = wl_registry_bind(
            wl_registry, name, &wl_compositor_interface, 4);
    }
}

int
main(int argc, char *argv[])
{
    struct our_state state = { 0 };
    // ...
    wl_registry_add_listener(registry, &registry_listener, &state);
    // ...
}
```

注意，我们在调用 wl_registry_bind 时指定了版本 4，这是撰写本书时的最新版本。拿到这个引用之后，我们就可以创建一个 wl_surface 了：

```
struct wl_surface *surface =
wl_compositor_create_surface(state.compositor);
```

在能够呈现它之前，我们必须先给它附加一个像素来源：一个 `wl_buffer`。

共享内存缓冲区

把像素从客户端传到合成器的最简单方式，也是唯一被写入 `wayland.xml` 的方式，就是 `wl_shm`——共享内存。简单地说，它允许你传递一个文件描述符，让合成器用 `MAP_SHARED` 进行 `mmap`，然后从这个池中共享出像素缓冲区。再加上一些简单的同步原语，以免大家为每个缓冲区争抢，你就得到了一个可行——而且可移植——的解决方案。

绑定 `wl_shm`

第 5.1 节讲解的注册表全局对象监听器会在 `wl_shm` 全局对象可用时通告它。绑定它相当直接。把第 5.1 节给出的示例加以扩展，就得到如下代码：

```
struct our_state {
    // ...
    struct wl_shm *shm;
    // ...
};

static void
registry_handle_global(void *data, struct wl_registry *registry,
                       uint32_t name, const char *interface, uint32_t
version)
{
    struct our_state *state = data;
    if (strcmp(interface, wl_shm_interface.name) == 0) {
        state->shm = wl_registry_bind(
            registry, name, &wl_shm_interface, 1);
    }
}

int
main(int argc, char *argv[])
{
    struct our_state state = { 0 };
    // ...
    wl_registry_add_listener(registry, &registry_listener, &state);
    // ...
}
```

绑定之后，我们可以选择通过 `wl_shm_add_listener` 添加一个监听器。合成器会通过这个监听器通告它支持的像素格式。所有可能的像素格式完整列表见 `wayland.xml`。有

两种格式是必须支持的：ARGB8888 和 XRGB8888，它们分别是带 alpha 通道和不带 alpha 通道的 24 位色。

分配共享内存池

结合 POSIX 的 `shm_open` 与随机文件名，可以用来创建适合此用途的文件，再用 `ftruncate` 把它调整到合适的大小。以下样板代码可在公有领域或 CC0 下自由使用：

```

#define _POSIX_C_SOURCE 200112L
#include <errno.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <time.h>
#include <unistd.h>

static void
randname(char *buf)
{
    struct timespec ts;
    clock_gettime(CLOCK_REALTIME, &ts);
    long r = ts.tv_nsec;
    for (int i = 0; i < 6; ++i) {
        buf[i] = 'A' + (r & 15) + (r & 16) * 2;
        r >>= 5;
    }
}

static int
create_shm_file(void)
{
    int retries = 100;
    do {
        char name[] = "/wl_shm-XXXXXX";
        randname(name + sizeof(name) - 7);
        --retries;
        int fd = shm_open(name, O_RDWR | O_CREAT | O_EXCL,
0600);
        if (fd >= 0) {
            shm_unlink(name);
            return fd;
        }
    } while (retries > 0 && errno == EEXIST);
    return -1;
}

int
allocate_shm_file(size_t size)
{
    int fd = create_shm_file();
    if (fd < 0)
        return -1;
    int ret;
    do {
        ret = ftruncate(fd, size);
    } while (ret < 0 && errno == EINTR);
    if (ret < 0) {
        close(fd);
        return -1;
    }
    return fd;
}

```

但愿这段代码足够不言自明（说说而已，常被打脸）。有了它，客户端就能相当轻松地创建一个共享内存池。比方说，我们想显示一个 1920x1080 的窗口。双缓冲需要两个缓冲区，也就是 4,147,200 个像素。假设像素格式是 WL_SHM_FORMAT_XRGB8888，那么每个像素 4 字节，池的总大小就是 16,588,800 字节。按照第 5.1 节的说明从注册表绑定 wl_shm 全局对象，然后像下面这样创建一个能容纳这些缓冲区的 shm 池：

```
const int width = 1920, height = 1080;
const int stride = width * 4;
const int shm_pool_size = height * stride * 2;

int fd = allocate_shm_file(shm_pool_size);
uint8_t *pool_data = mmap(NULL, shm_pool_size,
    PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

struct wl_shm *shm = ...; // Bound from registry
struct wl_shm_pool *pool = wl_shm_create_pool(shm, fd, shm_pool_size);
```

从池中创建缓冲区

等这个消息传到合成器，它也会对同一个文件描述符进行 mmap。不过 Wayland 是异步的，所以我们可以立即开始从这个池中分配缓冲区。既然我们为两个缓冲区分配了空间，就可以给每个分配一个索引，并把该索引转换成池中的字节偏移量。有了这些信息，我们就能创建一个 wl_buffer：

```
int index = 0;
int offset = height * stride * index;
struct wl_buffer *buffer = wl_shm_pool_create_buffer(pool, offset,
    width, height, stride, WL_SHM_FORMAT_XRGB8888);
```

现在我们可以往这个缓冲区里写图像了。例如，把它设置为纯白色：

```
uint32_t *pixels = (uint32_t *)&pool_data[offset];
memset(pixels, 0, width * height * 4);
```

或者，来点更有意思的，这是一个棋盘格图案：

```

uint32_t *pixels = (uint32_t *)&pool_data[offset];
for (int y = 0; y < height; ++y) {
    for (int x = 0; x < width; ++x) {
        if ((x + y / 8 * 8) % 16 < 8) {
            pixels[y * width + x] = 0xFF666666;
        } else {
            pixels[y * width + x] = 0xFFEEEEEE;
        }
    }
}
}

```

舞台已经搭好，我们把缓冲区附加到表面上，把整个表面标记为损坏¹，然后提交：

```

wl_surface_attach(surface, buffer, 0, 0);
wl_surface_damage(surface, 0, 0, UINT32_MAX, UINT32_MAX);
wl_surface_commit(surface);

```

如果你把所有这些刚学到的知识应用到亲自编写一个 Wayland 客户端上，那么到达这一步时，你可能会困惑于自己的缓冲区为什么没有显示在屏幕上。我们漏掉了关键的收尾步骤——为你的表面分配一个角色。

服务端的 wl_shm

不过，在抵达那一步之前，服务端这部分值得一记。libwayland 提供了一些辅助函数，让 wl_shm 的使用更简单。要为你的 display 配置它，只需要下面这些：

```

int
wl_display_init_shm(struct wl_display *display);

uint32_t *
wl_display_add_shm_format(struct wl_display *display, uint32_t
format);

```

前者创建全局对象并搭建好内部实现，后者添加一种受支持的像素格式（记得至少添加 ARGB8888 和 XRGB8888）。一旦客户端把缓冲区附加到它的某个表面上，你就可以把缓冲区资源传入 wl_shm_buffer_get，以获取一个 wl_shm_buffer 引用，并像下面这样使用它：

```
void
wl_shm_buffer_begin_access(struct wl_shm_buffer *buffer);

void
wl_shm_buffer_end_access(struct wl_shm_buffer *buffer);

void *
wl_shm_buffer_get_data(struct wl_shm_buffer *buffer);

int32_t
wl_shm_buffer_get_stride(struct wl_shm_buffer *buffer);

uint32_t
wl_shm_buffer_get_format(struct wl_shm_buffer *buffer);

int32_t
wl_shm_buffer_get_width(struct wl_shm_buffer *buffer);

int32_t
wl_shm_buffer_get_height(struct wl_shm_buffer *buffer);
```

如果你用 `begin_access` 和 `end_access` 来保护对缓冲区数据的访问，libwayland 会替你处理加锁。

1. “损坏”意为“该区域需要重绘” [↩](#)

Linux dmabuf

大多数 Wayland 合成器都在 GPU 上进行渲染，许多 Wayland 客户端同样在 GPU 上渲染。在这种情况下，采用共享内存的方式把缓冲区从客户端发送给合成器就非常低效了，因为客户端必须把数据从 GPU 读取到 CPU，然后合成器又得把它从 CPU 读回 GPU 才能进行渲染。

Linux DRM（直接渲染管理器，Direct Rendering Manager）接口（一些 BSD 系统上也实现了它）为我们提供了一种导出 GPU 资源句柄的手段。Mesa 是 Linux 用户空间图形驱动最主要的实现，它实现了一个协议，允许 EGL 用户把其 GPU 缓冲区的句柄从客户端传输给合成器以供渲染，全程无需把数据复制到 CPU。

该协议内部的工作原理超出了本书的范围，更适合在专门聚焦 Mesa 或 Linux DRM 的资料中详述。不过，我们可以对它的用法做一个简要总结：

1. 将 `eglGetPlatformDisplayEXT` 与 `EGL_PLATFORM_WAYLAND_KHR` 配合使用，创建一个 EGL display。
2. 按常规方式配置该 display，选择适合你实际情况的配置，并将 `EGL_SURFACE_TYPE` 设置为 `EGL_WINDOW_BIT`。
3. 使用 `wl_egl_window_create` 为给定的 `wl_surface` 创建一个 `wl_egl_window`。
4. 使用 `eglCreatePlatformWindowSurfaceEXT` 为 `wl_egl_window` 创建一个 `EGLSurface`。
5. 之后照常使用 EGL 即可，例如用 `eglMakeCurrent` 使你的表面的 EGL 上下文成为当前上下文，用 `eglSwapBuffers` 把最新的缓冲区发送给合成器并提交表面。

如果之后需要更改 `wl_egl_window` 的大小，可以使用 `wl_egl_window_resize`。

但我是真的想了解内部原理

一些不使用 libwayland 的 Wayland 程序员抱怨这种做法把 Mesa 和 libwayland 紧紧捆绑在了一起，事实也确实如此。然而，把二者解耦并非不可能——只是这需要你付出大量工作，也就是自己实现 linux-dmabuf。协议的细节请查阅 Wayland 扩展 XML；Mesa 的实现则位于 `src/egl/drivers/dri2/platform_wayland.c`（截至撰写时）。祝你好运，一路顺风。

对于服务端

遗憾的是，合成器一侧的细节既复杂又超出了本书的范围。不过，我可以为你指个方向：wlroots 的实现（截至撰写时位于 `types/wlr_linux_dmabuf_v1.c`）简明直观，应该能帮你走上正轨。

表面角色

我们已经创建了一个像素缓冲区，把它发送给了服务端，并把它附加到了一个表面上——据说我们就能通过这个表面把内容呈现给用户。然而，还缺少关键的一环来赋予这个表面意义：它的角色。

像素缓冲区可能在许多不同的情形下呈现给用户，而每一种情形都呼唤不同的语义。应用程序窗口固然是其中之一，但其他例子还包括光标图像或桌面壁纸。不妨对比一下应用程序窗口与光标的语义：光标无法被最小化，而应用程序窗口也不应该黏在鼠标上。为此，角色提供了另一层抽象，使你能够为表面赋予恰当的语义。

本书进行到第六章，你最迫不及待想分配给它的角色，多半就是应用程序窗口了。下一章将介绍实现这一目标的机制：XDG shell。

XDG shell 基础

XDG (cross-desktop group, 跨桌面组) shell 是 Wayland 的一个标准协议扩展, 它描述了应用窗口的语义。它定义了两个 `wl_surface` 角色: “`toplevel`” (顶层窗口), 用于你的顶层应用窗口; 以及 “`popup`” (弹出窗口), 用于上下文菜单、下拉菜单、工具提示之类的东西——它们是顶层窗口的子级。把它们组合起来, 你就能构成一棵表面树, 顶层是顶层窗口, 叶子则是弹出窗口或附加的顶层窗口。该协议还定义了一个 定位器接口, 它用于在只掌握窗口周围事物有限信息的情况下帮助定位弹出窗口。

`xdg-shell` 作为一个协议扩展, 并未定义在 `wayland.xml` 中。你会在 `wayland-protocols` 包里找到它。在你的系统上, 它大概安装在类似 `/usr/share/wayland-protocols/stable/xdg-shell/xdg-shell.xml` 这样的路径下。

`xdg_wm_base`

`xdg_wm_base` 是该规范定义的唯一全局对象, 它提供了用于创建你所需的其他各个对象的请求。最基本的实现从处理 “`ping`” 事件开始——当合成器发送它时, 你应当及时用一个 “`pong`” 请求来回应, 以表明你并没有陷入死锁。另一个请求用于创建定位器, 也就是前面提到的那些对象, 我们把关于它们的细节留到第 10 章。我们首先想深入了解的请求是 `get_xdg_surface`。

XDG 表面

在 xdg-shell 领域里，表面被称为 `xdg_surfaces`，这个接口带来了两类 XDG 表面——顶层窗口和弹出窗口——共有的一小部分功能。每一类 XDG 表面的语义仍然差异很大，因此它们必须通过一个额外的角色来显式指定。

`xdg_surface` 接口提供了额外的请求，用于分配更具体的弹出窗口和顶层窗口角色。一旦我们把一个对象绑定到 `xdg_wm_base` 全局对象上，就可以用 `get_xdg_surface` 请求来为一个 `wl_surface` 获取一个 `xdg_surface`。

```
<request name="get_xdg_surface">
  <arg name="id" type="new_id" interface="xdg_surface"/>
  <arg name="surface" type="object" interface="wl_surface"/>
</request>
```

`xdg_surface` 接口除了提供为你的表面分配更具体的顶层窗口或弹出窗口角色的请求之外，还包含了两类角色共有的一些重要功能。在我们继续介绍顶层窗口和弹出窗口专属的语义之前，先来回顾一下这些功能。

```
<event name="configure">
  <arg name="serial" type="uint" summary="serial of the configure event"/>
</event>

<request name="ack_configure">
  <arg name="serial" type="uint" summary="the serial from the configure event"/>
</request>
```

`xdg-surface` 最重要的 API 就是这一对：`configure` 和 `ack_configure`。你可能还记得，Wayland 的一个目标就是让每一帧都完美无缺。这意味着不会有一帧带着只应用了一半的状态变更被显示出来，而为了实现这一点，我们必须让这些变更在客户端和服务端之间保持同步。对于 XDG 表面而言，这对消息就是支撑这一点的机制。

我们现在只覆盖基础内容，所以就概括这两个事件的重要性：当来自服务端的这些事件告知你某个表面的配置（或重新配置）时，把它们应用到一份待定状态上。当一个 `configure` 事件到达时，应用待定的变更，用 `ack_configure` 确认你已完成，然后渲染并提交一个新帧。我们会在下一章中实际演示这一点，并在第 8.1 节详细解释。

```
<request name="set_window_geometry">
  <arg name="x" type="int"/>
  <arg name="y" type="int"/>
  <arg name="width" type="int"/>
  <arg name="height" type="int"/>
</request>
```

`set_window_geometry` 请求主要用于使用客户端侧装饰（decoration）的应用，用来区分它们的表面中哪些部分被视为窗口的一部分，哪些部分不是。最常见的情况是，它被用来把渲染在窗口背后的客户端侧投影从窗口的一部分中排除出去。合成器可能会运用这一信息来管理它自己安排窗口、与窗口交互的行为。

应用窗口

我们为了走到这一步已经剥了不知多少层（费了不知多少功夫），但时候到了：XDG 顶层窗口就是我们将最终用来显示应用窗口的接口。XDG 顶层窗口接口有许多用于管理应用窗口的请求和事件，包括处理最小化和最大化状态、设置窗口标题等等。我们会在后续章节中详细讨论它的每一部分，所以现在只关注基础内容。

基于上一章的知识，我们知道可以从一个 `wl_surface` 得到一个 `xdg_surface`，但这只构成了第一步：把一个表面纳入 XDG shell 的体系。下一步是把那个 XDG 表面变成一个 XDG 顶层窗口——一个“顶层”应用窗口，它因处于我们最终会用 XDG shell 创建的窗口与弹出菜单层级结构的顶层而得名。要创建其中一个，我们可以使用 `xdg_surface` 接口中相应的请求：

```
<request name="get_toplevel">
  <arg name="id" type="new_id" interface="xdg_toplevel"/>
</request>
```

这个新的 `xdg_toplevel` 接口为我们提供了许多请求和事件，用于管理应用窗口的生命周期。第 10 章会深入探讨这些内容，但我知道你已经迫不及待想让某个东西出现在屏幕上了。如果你遵循以下步骤，处理上一章讨论过的 XDG 表面的 `configure` 和 `ack_configure` 装置，并把一个 `wl_buffer` attach（附加）并提交到我们的 `wl_surface` 上，那么一个应用窗口就会出现，并把你的缓冲区内容呈现给用户。下一章提供了正是这样做的示例代码。它还利用了一个我们尚未覆盖的额外 XDG 顶层窗口请求：

```
<request name="set_title">
  <arg name="title" type="string"/>
</request>
```

这个应该相当不言自明。还有一个类似的、我们没有在示例代码中使用，但可能适合你的应用的请求：

```
<request name="set_app_id">
  <arg name="app_id" type="string"/>
</request>
```

标题通常显示在窗口装饰、任务栏等地方，而 app ID 则用于标识你的应用，或把你的窗口归为一组。你可以这样使用它们：把窗口标题设为 “Application windows — The Wayland Protocol — Firefox”，把你的 app ID 设为 “firefox”。

总而言之，以下步骤将带你从零开始把一个窗口显示在屏幕上：

1. 绑定到 `wl_compositor`，并用它创建一个 `wl_surface`。
2. 绑定到 `xdg_wm_base`，并用它和你的 `wl_surface` 创建一个 `xdg_surface`。
3. 用 `xdg_surface.get_toplevel` 从 `xdg_surface` 创建一个 `xdg_toplevel`。
4. 为 `xdg_surface` 配置一个监听器，并等待 `configure` 事件。
5. 绑定到你选定的缓冲区分配机制（例如 `wl_shm`）并分配一个共享缓冲区，然后把你的内容渲染到其中。
6. 使用 `wl_surface.attach` 把 `wl_buffer` attach 到 `wl_surface`。
7. 使用 `xdg_surface.ack_configure`，把来自 `configure` 的 `serial` 传给它，确认你已经准备好了一个合适的帧。
8. 发送一个 `wl_surface.commit` 请求。

翻到下一页看看这些步骤的实际运用。

扩展示例代码

运用我们到目前为止学到的全部内容，我们现在可以编写一个能在屏幕上显示东西的 Wayland 客户端了。下面的代码是一个完整的 Wayland 应用，它打开一个 XDG 顶层窗口窗口，并在上面显示一个 640x480 的方块网格。像这样编译它：

```
wayland-scanner private-code \  
  < /usr/share/wayland-protocols/stable/xdg-shell/xdg-shell.xml \  
  > xdg-shell-protocol.c  
wayland-scanner client-header \  
  < /usr/share/wayland-protocols/stable/xdg-shell/xdg-shell.xml \  
  > xdg-shell-client-protocol.h  
cc -o client client.c xdg-shell-protocol.c -lwayland-client -lrt
```

然后运行 `./client` 来观看它的实际效果，或者用 `WAYLAND_DEBUG=1 ./client` 来带上许多有用的调试信息。嗒哒！在后续章节中，我们会在这个客户端的基础上继续构建，所以把这段代码妥善收好。

```

#define _POSIX_C_SOURCE 200112L
#include <errno.h>
#include <fcntl.h>
#include <limits.h>
#include <stdbool.h>
#include <string.h>
#include <sys/mman.h>
#include <time.h>
#include <unistd.h>
#include <wayland-client.h>
#include "xdg-shell-client-protocol.h"

/* Shared memory support code */
static void
randname(char *buf)
{
    struct timespec ts;
    clock_gettime(CLOCK_REALTIME, &ts);
    long r = ts.tv_nsec;
    for (int i = 0; i < 6; ++i) {
        buf[i] = 'A' + (r & 15) + (r & 16) * 2;
        r >>= 5;
    }
}

static int
create_shm_file(void)
{
    int retries = 100;
    do {
        char name[] = "/wl_shm-XXXXXX";
        randname(name + sizeof(name) - 7);
        --retries;
        int fd = shm_open(name, O_RDWR | O_CREAT | O_EXCL, 0600);
        if (fd >= 0) {
            shm_unlink(name);
            return fd;
        }
    } while (retries > 0 && errno == EEXIST);
    return -1;
}

static int
allocate_shm_file(size_t size)
{
    int fd = create_shm_file();
    if (fd < 0)
        return -1;
    int ret;
    do {
        ret = ftruncate(fd, size);
    } while (ret < 0 && errno == EINTR);
    if (ret < 0) {
        close(fd);
    }
}

```

```

        return -1;
    }
    return fd;
}

/* Wayland code */
struct client_state {
    /* Globals */
    struct wl_display *wl_display;
    struct wl_registry *wl_registry;
    struct wl_shm *wl_shm;
    struct wl_compositor *wl_compositor;
    struct xdg_wm_base *xdg_wm_base;
    /* Objects */
    struct wl_surface *wl_surface;
    struct xdg_surface *xdg_surface;
    struct xdg_toplevel *xdg_toplevel;
};

static void
wl_buffer_release(void *data, struct wl_buffer *wl_buffer)
{
    /* Sent by the compositor when it's no longer using this buffer */
    wl_buffer_destroy(wl_buffer);
}

static const struct wl_buffer_listener wl_buffer_listener = {
    .release = wl_buffer_release,
};

static struct wl_buffer *
draw_frame(struct client_state *state)
{
    const int width = 640, height = 480;
    int stride = width * 4;
    int size = stride * height;

    int fd = allocate_shm_file(size);
    if (fd == -1) {
        return NULL;
    }

    uint32_t *data = mmap(NULL, size,
        PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    if (data == MAP_FAILED) {
        close(fd);
        return NULL;
    }

    struct wl_shm_pool *pool = wl_shm_create_pool(state->wl_shm, fd,
size);
    struct wl_buffer *buffer = wl_shm_pool_create_buffer(pool, 0,
        width, height, stride, WL_SHM_FORMAT_XRGB8888);
    wl_shm_pool_destroy(pool);
    close(fd);
}

```

```

/* Draw checkered background */
for (int y = 0; y < height; ++y) {
    for (int x = 0; x < width; ++x) {
        if ((x + y / 8 * 8) % 16 < 8)
            data[y * width + x] = 0xFF666666;
        else
            data[y * width + x] = 0xFFEEEEEE;
    }
}

munmap(data, size);
wl_buffer_add_listener(buffer, &wl_buffer_listener, NULL);
return buffer;
}

static void
xdg_surface_configure(void *data,
                      struct xdg_surface *xdg_surface, uint32_t serial)
{
    struct client_state *state = data;
    xdg_surface_ack_configure(xdg_surface, serial);

    struct wl_buffer *buffer = draw_frame(state);
    wl_surface_attach(state->wl_surface, buffer, 0, 0);
    wl_surface_commit(state->wl_surface);
}

static const struct xdg_surface_listener xdg_surface_listener = {
    .configure = xdg_surface_configure,
};

static void
xdg_wm_base_ping(void *data, struct xdg_wm_base *xdg_wm_base, uint32_t
serial)
{
    xdg_wm_base_pong(xdg_wm_base, serial);
}

static const struct xdg_wm_base_listener xdg_wm_base_listener = {
    .ping = xdg_wm_base_ping,
};

static void
registry_global(void *data, struct wl_registry *wl_registry,
                uint32_t name, const char *interface, uint32_t version)
{
    struct client_state *state = data;
    if (strcmp(interface, wl_shm_interface.name) == 0) {
        state->wl_shm = wl_registry_bind(
            wl_registry, name, &wl_shm_interface, 1);
    } else if (strcmp(interface, wl_compositor_interface.name) == 0) {
        state->wl_compositor = wl_registry_bind(
            wl_registry, name, &wl_compositor_interface, 4);
    } else if (strcmp(interface, xdg_wm_base_interface.name) == 0) {

```

```

        state->xdg_wm_base = wl_registry_bind(
            wl_registry, name, &xdg_wm_base_interface, 1);
        xdg_wm_base_add_listener(state->xdg_wm_base,
            &xdg_wm_base_listener, state);
    }
}

static void
registry_global_remove(void *data,
    struct wl_registry *wl_registry, uint32_t name)
{
    /* This space deliberately left blank */
}

static const struct wl_registry_listener wl_registry_listener = {
    .global = registry_global,
    .global_remove = registry_global_remove,
};

int
main(int argc, char *argv[])
{
    struct client_state state = { 0 };
    state.wl_display = wl_display_connect(NULL);
    state.wl_registry = wl_display_get_registry(state.wl_display);
    wl_registry_add_listener(state.wl_registry, &wl_registry_listener,
&state);
    wl_display_roundtrip(state.wl_display);

    state.wl_surface =
wl_compositor_create_surface(state.wl_compositor);
    state.xdg_surface = xdg_wm_base_get_xdg_surface(
        state.xdg_wm_base, state.wl_surface);
    xdg_surface_add_listener(state.xdg_surface, &xdg_surface_listener,
&state);
    state.xdg_toplevel = xdg_surface_get_toplevel(state.xdg_surface);
    xdg_toplevel_set_title(state.xdg_toplevel, "Example client");
    wl_surface_commit(state.wl_surface);

    while (wl_display_dispatch(state.wl_display)) {
        /* This space deliberately left blank */
    }

    return 0;
}

```

深入探讨表面

到目前为止，我们展示过的表面接口的基本方面足以向用户呈现数据，但表面接口还提供了许多额外的请求和事件，以便更高效地使用。许多——即便不是大多数——应用程序并不需要在每一帧都重绘整个表面。甚至连决定何时绘制下一帧，也最好在合成器的协助下完成。在本章中，我们将深入探讨 `wl_surface` 的各项特性。

表面生命周期

我们之前提到过，Wayland 在设计上会以原子方式更新一切，因此任何一帧都不会以无效或中间状态呈现。在可为应用程序窗口及其他表面配置的众多属性之中，这一原子性背后的驱动机制正是 `wl_surface` 本身。

每个表面都有一个 *pending*（待定）状态和一个 *applied*（已应用）状态，而在刚创建时则完全没有状态。*pending* 状态是在任意数量的客户端请求与服务端事件的过程中协商形成的；当双方都认可它代表一个一致的表面状态时，该表面即被提交——*pending* 状态随之被应用于表面的当前状态。在此之前，合成器会继续渲染上一个一致的状态；一旦提交，便会从下一帧起使用新状态。

以原子方式更新的状态包括：

- 已附加的 `wl_buffer`，即构成表面内容的像素
- 自上一帧以来遭到“damage”（损伤）而需要重绘的区域
- 接受输入事件的区域
- 被视为不透明的区域¹
- 作用于已附加 `wl_buffer` 的变换，用于旋转缓冲区或仅呈现其中一部分
- 缓冲区的缩放因子，用于 HiDPI 显示器

除了表面的这些特性之外，表面的角色也可能拥有类似的双缓冲状态。所有这些状态，连同与该角色相关联的任何状态，都会在发送 `wl_surface.commit` 时被应用。如果你改变了主意，可以多次发送这些请求；而当表面最终被提交时，这些属性中每一项都只会采用最近一次的值。

当你首次创建表面时，其初始状态是无效的。要让它变为有效（亦即映射（map）该表面），你需要提供建立该表面第一个一致状态所需的必要信息。这包括为它指定一个角色（比如 `xdg_toplevel`）、分配并附加一个缓冲区，以及为该表面配置任何角色特定的状态。当你发送 `wl_surface.commit` 且上述状态均已正确配置时，该表面就变为有效（即已映射（mapped）），并将由合成器予以呈现。

下一个问题是：我该在何时准备新的一帧？

1. 合成器将其用于优化目的。↩

帧回调

更新表面最简单的方式，就是在它需要变化时直接重新渲染并附着新帧。比如，这种方式非常适合事件驱动的应用程序。用户按下一个键，文本框需要重新渲染，于是你可以立即重新渲染它、损伤相应的区域，并附着一个新的缓冲区，让它在下一帧被呈现。

然而，有些应用程序可能希望连续渲染帧。你可能在渲染电子游戏的帧、播放视频，或者渲染动画。你的显示器具有固有的刷新率，也就是它能够显示更新的最快速度（通常是一个类似 60 Hz、144 Hz 之类的数字）。以高于此的速度渲染帧毫无意义，而且这样做会浪费资源——CPU、GPU，甚至用户的电量。如果你在每次显示刷新之间发送若干帧，那么除最后一帧外都会被丢弃，白白渲染了。

此外，合成器甚至可能根本不想为你显示新的帧。你的应用程序可能处于屏幕之外、被最小化，或者被其他窗口遮挡；又或者只显示你应用程序的一小块缩略图，因此它们可能希望以更低的帧率渲染你，以节省资源。出于这个原因，在 Wayland 客户端中连续渲染帧的最佳方式，是让合成器告诉你它何时准备好接收新帧：也就是使用帧回调。

```
<interface name="wl_surface" version="4">
  <!-- ... -->

  <request name="frame">
    <arg name="callback" type="new_id" interface="wl_callback" />
  </request>

  <!-- ... -->
</interface>
```

这个请求会分配一个 `wl_callback` 对象，它的接口相当简单：

```
<interface name="wl_callback" version="1">
  <event name="done">
    <arg name="callback_data" type="uint" />
  </event>
</interface>
```

当你在一个表面上请求帧回调时，一旦合成器为该表面准备好接收新帧，它就会向该回调对象发送一个 `done` 事件。对于 `frame` 事件而言，`callback_data` 被设置为当前时间，单位为毫秒，其纪元未指定。你可以将它与你上一帧的时间进行比较，从而计算动画的进度或缩放输入事件。¹

有了帧回调这件利器，我们何不改造第 7.3 节的应用程序，让它每一帧都滚动一点呢？让我们先往 `client_state` 结构体里添加一点状态：

```

--- a/client.c
+++ b/client.c
@@ -71,6 +71,8 @@ struct client_state {
    struct xdg_surface *xdg_surface;
    struct xdg_toplevel *xdg_toplevel;
+
+    /* State */
+    float offset;
+    uint32_t last_frame;
};

static void wl_buffer_release(void *data, struct wl_buffer
*wl_buffer) {

```

然后我们更新 `draw_frame` 函数，让它把偏移量考虑进去：

```

@@ -107,9 +109,10 @@ draw_frame(struct client_state *state)
    close(fd);

    /* Draw checkered background */
+    int offset = (int)state->offset % 8;
    for (int y = 0; y < height; ++y) {
        for (int x = 0; x < width; ++x) {
-            if ((x + y / 8 * 8) % 16 < 8)
+            if (((x + offset) + (y + offset) / 8 * 8) % 16
< 8)

                data[y * width + x] = 0xFF666666;
            else
                data[y * width + x] = 0xFFEEEEEE;

```

在 `main` 函数中，让我们为我们的第一个新帧注册一个回调：

```

@@ -195,6 +230,9 @@ main(int argc, char *argv[])
    xdg_toplevel_set_title(state.xdg_toplevel, "Example client");
    wl_surface_commit(state.wl_surface);

+    struct wl_callback *cb = wl_surface_frame(state.wl_surface);
+    wl_callback_add_listener(cb, &wl_surface_frame_listener,
&state);
+
    while (wl_display_dispatch(state.wl_display)) {
        /* This space deliberately left blank */
    }

```

然后像这样实现它：

```

@@ -147,6 +150,38 @@ static const struct xdg_wm_base_listener
xdg_wm_base_listener = {
    .ping = xdg_wm_base_ping,
};

+static const struct wl_callback_listener wl_surface_frame_listener;
+
+static void
+wl_surface_frame_done(void *data, struct wl_callback *cb, uint32_t
time)
+{
+    /* Destroy this callback */
+    wl_callback_destroy(cb);
+
+    /* Request another frame */
+    struct client_state *state = data;
+    cb = wl_surface_frame(state->wl_surface);
+    wl_callback_add_listener(cb, &wl_surface_frame_listener,
state);
+
+    /* Update scroll amount at 24 pixels per second */
+    if (state->last_frame != 0) {
+        int elapsed = time - state->last_frame;
+        state->offset += elapsed / 1000.0 * 24;
+    }
+
+    /* Submit a frame for this event */
+    struct wl_buffer *buffer = draw_frame(state);
+    wl_surface_attach(state->wl_surface, buffer, 0, 0);
+    wl_surface_damage_buffer(state->wl_surface, 0, 0, INT32_MAX,
INT32_MAX);
+    wl_surface_commit(state->wl_surface);
+
+    state->last_frame = time;
+}
+
+static const struct wl_callback_listener wl_surface_frame_listener =
{
+    .done = wl_surface_frame_done,
+};
+
+static void
registry_global(void *data, struct wl_registry *wl_registry,
uint32_t name, const char *interface, uint32_t
version)

```

现在，每一帧我们都会：

1. 销毁刚刚用过的帧回调。
2. 为下一帧请求一个新的回调。
3. 渲染并提交新帧。

将第三步拆解开来就是：

1. 用一个新偏移量更新我们的状态，利用自上一帧以来的时间以恒定的速率滚动。
2. 准备一个新的 `wl_buffer`，并为它渲染一帧。
3. 将这个新的 `wl_buffer` 附着到我们的表面上。
4. 损伤整个表面。
5. 提交表面。

第 3 步和第 4 步更新了该表面的待定状态，给它提供了新的缓冲区，并表明整个表面都已改变。第 5 步提交这个待定状态，将其应用到表面的当前状态，并在下一帧使用它。以原子方式应用这个新缓冲区，意味着我们绝不会显示出上一帧的一半，从而带来流畅无撕裂的体验。编译并运行更新后的客户端，亲自试一试吧！

-
1. 想要更精确的计时？在第 12.1 节中，我们将讨论一个协议扩展，它以纳秒精度告诉你每一帧究竟是在何时呈现给用户的。 [↩](#)

表面损伤

你可能已经注意到，在上一个示例中，当我们为表面提交一个新帧时，添加了这样一行代码：

```
wl_surface_damage_buffer(state->wl_surface, 0, 0, INT32_MAX,
INT32_MAX);
```

如果注意到了，真是好眼力！这段代码损伤了我们的表面，向合成器表明它需要被重绘。这里我们损伤了整个表面（而且远远超出了它），但我们其实也可以只损伤它的一部分。

举例来说，假设你编写了一个 GUI 工具包，用户正在一个文本框中打字。那个文本框可能只占据窗口的一小部分，而每个新字符占据的部分更小。当用户按下一个键时，你可以只渲染追加到其正在书写文本之后的新字符，然后只损伤表面的那一部分。随后合成器可以只复制你表面的一小部分，这能显著加快速度——尤其是在嵌入式设备上。当你在字符之间闪烁光标时，你会希望为它的更新提交损伤；而当用户切换视图时，你很可能损伤整个表面。这样一来，每个人都少做了工作，用户也会因为你改善了他的电池续航而感谢你。

注意：Wayland 协议提供了两个用于损伤表面的请求：`damage` 和 `damage_buffer`。前者实际上已被弃用，你应当只使用后者。它们之间的区别在于，`damage` 会考虑所有影响该表面的变换，例如旋转、缩放因子，以及缓冲区位置和裁剪。后者则改为相对于缓冲区来施加损伤，这通常更容易推理。

表面区域

我们已经使用过 `wl_compositor` 接口，通过 `wl_compositor.create_surface` 创建 `wl_surface`。不过请注意，它还有第二个请求：`create_region`。

```
<interface name="wl_compositor" version="4">
  <request name="create_surface">
    <arg name="id" type="new_id" interface="wl_surface" />
  </request>

  <request name="create_region">
    <arg name="id" type="new_id" interface="wl_region" />
  </request>
</interface>
```

`wl_region` 接口定义了一组矩形，它们共同构成一个任意形状的几何区域。它的请求允许你针对它所定义的几何形状执行按位运算——向其中添加矩形，或从中减去矩形。

```
<interface name="wl_region" version="1">
  <request name="destroy" type="destructor" />

  <request name="add">
    <arg name="x" type="int" />
    <arg name="y" type="int" />
    <arg name="width" type="int" />
    <arg name="height" type="int" />
  </request>

  <request name="subtract">
    <arg name="x" type="int" />
    <arg name="y" type="int" />
    <arg name="width" type="int" />
    <arg name="height" type="int" />
  </request>
</interface>
```

例如，要制作一个中间带洞的矩形，你可以：

1. 发送 `wl_compositor.create_region` 来分配一个 `wl_region` 对象。
2. 发送 `wl_region.add(0, 0, 512, 512)` 创建一个 512x512 的矩形。
3. 发送 `wl_region.subtract(128, 128, 256, 256)` 从该区域中央移除一个 256x256 的矩形。

这些区域也可以是彼此分离的；它不必是一个单一连续的多边形。一旦你创建了这样一个区域，就可以把它传入 `wl_surface` 接口，具体是通过 `set_opaque_region` 和 `set_input_region` 请求。

```
<interface name="wl_surface" version="4">
  <request name="set_opaque_region">
    <arg name="region" type="object" interface="wl_region" allow-
null="true" />
  </request>

  <request name="set_input_region">
    <arg name="region" type="object" interface="wl_region" allow-
null="true" />
  </request>
</interface>
```

不透明区域（opaque 区域）是给合成器的一个提示，指明你的表面的哪些部分被视为不透明。基于这一信息，合成器可以优化其渲染过程。例如，如果你的表面完全不透明，并且遮挡了它下面的另一个窗口，那么合成器就不会在你下面的窗口上浪费任何重绘的时间。默认情况下该区域为空，这意味着假定你的表面的任何部分都可能是透明的。这使默认情况成为效率最低但最正确的选择。

输入区域（input 区域）表明你的表面的哪些部分接受指针和触摸输入事件。例如，你可能在自己的表面下方绘制了投影，但发生在该区域内的输入事件应当被传递给位于你下方的客户端。又或者，如果你的窗口形状不寻常，你可以创建一个与之形状相符的输入区域。对于大多数表面类型而言，默认情况下你的整个表面都接受输入。

这两个请求都可以通过传入 null 而非 wl_region 对象来设置一个空区域。它们也都是双缓冲的——所以要发送一次 wl_surface.commit 才能让你的更改生效。一旦你用它发送过 set_opaque_region 或 set_input_region 请求，就可以销毁该 wl_region 对象以释放其资源。在这些请求发送之后再更新该区域，并不会更新表面的状态。

子表面

在核心 Wayland 协议 `wayland.xml` 中，只定义了一种¹ 表面角色：子表面。它们相对于父表面拥有 X、Y 位置——该位置不必受父表面边界的约束——并且相对于其同级表面和父表面拥有 Z 序。

这项特性的一些用例包括：以原生像素格式播放视频表面，并在其上叠加 RGBA 用户界面或字幕；为主应用程序界面使用一个 OpenGL 表面，并用子表面以软件方式渲染窗口装饰；或者在不需在客户端重绘的情况下移动界面的某些部分。借助硬件平面

（hardware plane），合成器甚至也可能无需为更新你的子表面而重绘任何内容。尤其在嵌入式系统上，当它契合你的用例时，这会特别有用。设计巧妙的应用程序可以利用子表面来获得极高的效率。

管理这些对象的接口是 `wl_subcompositor` 接口。`get_subsurface` 请求是通往子合成器（subcompositor）的主要入口：

```
<request name="get_subsurface">
  <arg name="id" type="new_id" interface="wl_subsurface" />
  <arg name="surface" type="object" interface="wl_surface" />
  <arg name="parent" type="object" interface="wl_surface" />
</request>
```

一旦你有了一个与某个 `wl_surface` 关联的 `wl_subsurface` 对象，该表面便成为其父表面的子表面。子表面自身还可以拥有子表面，从而在任何顶层窗口之下形成一棵有序的表面树。对这些子对象的操作通过 `wl_subsurface` 接口完成：

```
<request name="set_position">
  <arg name="x" type="int" summary="x coordinate in the parent surface"/>
  <arg name="y" type="int" summary="y coordinate in the parent surface"/>
</request>
```

```
<request name="place_above">
  <arg name="sibling" type="object" interface="wl_surface" />
</request>
```

```
<request name="place_below">
  <arg name="sibling" type="object" interface="wl_surface" />
</request>
```

```
<request name="set_sync" />
<request name="set_desync" />
```

子表面的 Z 序可以通过将它置于任一与其共享同一父表面的同级表面之上或之下，或置于父表面本身之上或之下，来加以改变。

对 `wl_subsurface` 各项属性的同步机制需要稍作解释。这些位置和 Z 序属性与父表面的生命周期同步。当为主表面发送 `wl_surface.commit` 请求时，其所有子表面的位置和 Z 序变更都会随之被应用。

然而，与该子表面关联的 `wl_surface` 状态——例如缓冲区的附加以及损伤的累积——不必与父表面的生命周期绑定。这正是 `set_sync` 和 `set_desync` 请求的用途。与父表面同步的子表面，会在父表面被提交时提交其全部状态。解除同步（`desync`）的表面则像别的表面一样管理自己的提交生命周期。

简而言之，`sync` 和 `desync` 请求是非缓冲的，会立即生效。位置和 Z 序请求则是缓冲的，且不受表面的 `sync/desync` 属性影响——它们始终随父表面一起提交。其余的表面状态（位于关联的 `wl_surface` 上）则依照该子表面的 `sync/desync` 状态进行提交。

1. 不考虑已弃用的 `wl_shell` 接口。↩

高密度表面（HiDPI）

在过去几年里，高端显示器的像素密度出现了巨大的飞跃，新式显示器在相同的物理面积内塞进了往年两倍之多的像素。我们把这类显示器称为“HiDPI”，即“high dots per inch”（每英寸高点数）的缩写。然而，这类显示器远远领先于它们的“LoDPI”同类，以至于必须做出应用程序层面的改变，才能恰当地利用它们。如果在相同的空间内把屏幕分辨率翻倍，而我们又不给予它们任何特殊考量，就会把所有用户界面的尺寸减半。对于大多数显示器而言，这会使文字无法阅读，并使交互元素小得令人不适。

不过作为交换，我们的矢量图形获得了大幅提升的图形保真度，尤其是在文字渲染方面。Wayland 通过为每个输出添加一个“缩放系数”（scale factor）来应对这一问题，并期望客户端将这个缩放系数应用到它们的界面上。此外，对 HiDPI 没有感知的客户端会以不作为的方式来表明这一局限，从而让合成器得以通过放大它们的缓冲区来弥补。合成器通过相应的事件通告每个输出的缩放系数：

```
<interface name="wl_output" version="3">
  <!-- ... -->
  <event name="scale" since="2">
    <arg name="factor" type="int" />
  </event>
</interface>
```

请注意，这是在第 2 版中加入的，因此在绑定 `wl_output` 全局对象时，你必须将版本设置为至少 2 才能接收到这些事件。然而，这还不足以决定在你的客户端中使用 HiDPI。要做出这个判断，合成器还必须为你的 `wl_surface` 发送 `enter` 事件，以表明它已经“进入”（即正显示在）某一个或某几个特定的输出上：

```
<interface name="wl_surface" version="4">
  <!-- ... -->
  <event name="enter">
    <arg name="output" type="object" interface="wl_output" />
  </event>
</interface>
```

一旦你知道了客户端所显示于其上的输出的集合，就应该取缩放系数中的最大值，把缓冲区的尺寸（以像素为单位）乘以这个值，然后以 2x 或 3x（或 Nx）的缩放来渲染用户界面。随后，像这样表明缓冲区是以何种缩放来准备的：

```
<interface name="wl_surface" version="4">
  <!-- ... -->
  <request name="set_buffer_scale" since="3">
    <arg name="scale" type="int" />
  </request>
</interface>
```

注意：这要求 `wl_surface` 的版本为 3 或更新。这也是你在绑定 `wl_compositor` 时应当传给 `wl_registry` 的版本号。

在下一次 `wl_surface.commit` 时，你的表面将采用这个缩放系数。如果它大于该表面所显示于其上的某个输出的缩放系数，合成器会将它缩小。如果它小于某个输出的缩放系数，合成器则会将它放大。

seat（座位）：处理输入

将应用程序显示给用户只是 I/O 等式的一半——大多数应用程序还需要处理输入。为此，seat 提供了一种对 Wayland 上输入事件的抽象。用哲学的话说，Wayland seat 指的是用户坐下来操作计算机的一个“座位”，它最多关联一个键盘和至多一个“指针”设备（即鼠标或触摸板）。触摸屏、绘图板设备等也定义了类似的关系。

务必记住，这只是一层抽象:Wayland 显示上呈现的 seat 未必与现实一一对应。在实践中，一个 Wayland 会话上很少会出现多于一个 seat 可用的情况。如果你给计算机插上第二个键盘，它通常会被分配到与第一个相同的 seat，键盘布局等等会随着你开始在其中一个上打字而动态切换。这些实现细节留给 Wayland 合成器去斟酌。

从客户端的角度来看，这件事相当直接。如果你绑定到 wl_seat 全局对象，就能访问以下接口：

```
<interface name="wl_seat" version="7">
  <enum name="capability" bitfield="true">
    <entry name="pointer" value="1" />
    <entry name="keyboard" value="2" />
    <entry name="touch" value="4" />
  </enum>

  <event name="capabilities">
    <arg name="capabilities" type="uint" enum="capability" />
  </event>

  <event name="name" since="2">
    <arg name="name" type="string" />
  </event>

  <request name="get_pointer">
    <arg name="id" type="new_id" interface="wl_pointer" />
  </request>

  <request name="get_keyboard">
    <arg name="id" type="new_id" interface="wl_keyboard" />
  </request>

  <request name="get_touch">
    <arg name="id" type="new_id" interface="wl_touch" />
  </request>

  <request name="release" type="destructor" since="5" />
</interface>
```

注意：这个接口已经更新过很多次——绑定全局对象时要注意版本。本书假定你绑定的是最新版本，在撰写本书时即版本 7。

这个接口相对简单直接。服务端会向客户端发送一个 `capabilities` 事件，以表明该 `seat` 支持哪些类型的输入设备——用 `capability` 值的位域表示——客户端据此可以绑定它想要使用的输入设备。例如，如果服务端发送的 `capabilities` 满足 `(caps & WL_SEAT_CAPABILITY_KEYBOARD) > 0`，客户端便可使用 `get_keyboard` 请求为该 `seat` 获取一个 `wl_keyboard` 对象。每种具体输入设备的语义将在后续章节中介绍。

在进入那些内容之前，我们先介绍一些通用的语义。

事件序列号

Wayland 客户端可能执行的某些操作，需要借助一种轻量形式的认证，即以输入事件序列号的形式。例如，打开弹出窗口的客户端（右键唤出的上下文菜单就是一种弹出窗口）可能希望在服务端“抓取”来自受影响 `seat` 的所有输入事件，直到该弹出窗口被关闭。为防止这一特性被滥用，服务端可以给它发送的每个输入事件分配一个序列号，并要求客户端在请求中包含其中一个序列号。

当服务端收到这样的请求时，它会查找与该序列号关联的输入事件并做出判断。如果该事件发生得太久远、或属于错误的表面、或不是正确的类型——例如，它可以拒绝在晃动鼠标时发起的抓取，却允许在点击时发起的抓取——它就可以拒绝该请求。

从服务端的角度看，它只需在发送每个输入事件时附带一个递增的整数，并记录下对特定用例而言被视为有效的序列号，以供日后校验。客户端从自己的输入事件处理器中接收到这些序列号，只需立即把它们回传，便能执行所需的操作。

我们将在后面的章节中更详细地讨论这些内容，那时我们会开始介绍需要输入事件序列号来校验的具体请求。

输入帧

出于实际原因，来自输入设备的单个输入事件可能被拆分成若干个 Wayland 事件。例如，当你使用滚轮时，`wl_pointer` 会发出一个 `axis` 事件，但它会另外发出一个事件，告诉你这是哪种轴：滚轮、触摸板上的手指、把滚轮向侧面拨动，等等。若用户在足够短的时间内同时做了这些动作，来自输入源的同一个输入事件也可能包含一些鼠标移动或按键操作。

这些相关事件在语义上的分组方式因输入类型而略有差异，但 `frame` 事件通常是它们共有的。简而言之，如果你把从某个设备收到的所有输入事件缓冲起来，然后等待 `frame`

事件表明你已收到单个输入“帧”的全部事件，你就能把这批缓冲的 *Wayland* 事件解读为一个单独的输入事件，然后重置缓冲区，开始收集下一帧的事件。

如果这听起来太复杂，不必担心。许多应用程序无需操心输入帧。只有当你开始进行更复杂的输入事件处理时，才需要关注这一点。

释放设备

当你用完某个设备后，每个接口都有一个 `release` 请求可供你清理它。它大致如下所示：

```
<request name="release" type="destructor" />
```

足够简单。

指针输入

借助 `wl_seat.get_pointer` 请求，客户端可以获得一个 `wl_pointer` 对象。每当用户移动指针、按下鼠标按键、使用滚轮等等时——只要指针位于你的某个表面之上——服务端就会向它发送事件。我们可以通过 `wl_pointer.enter` 事件来判断这一条件是否满足：

```
<event name="enter">
  <arg name="serial" type="uint" />
  <arg name="surface" type="object" interface="wl_surface" />
  <arg name="surface_x" type="fixed" />
  <arg name="surface_y" type="fixed" />
</event>
```

当指针移到我们的某个表面之上时，服务端会发送此事件，并同时指明被“进入”的表面，以及指针所处的表面局部坐标（自左上角算起）。这里的坐标用“fixed”类型表示，你可能还记得第 2.1 节里说过它表示一个 24.8 位定点精度数（`wl_fixed_to_double` 会把它转换为 C 的 `double` 类型）。

当指针移离你的表面时，对应的事件更简短：

```
<event name="leave">
  <arg name="serial" type="uint" />
  <arg name="surface" type="object" interface="wl_surface" />
</event>
```

一旦指针进入了你的表面，你就会开始收到针对它的额外事件，我们稍后会讨论。不过，你很可能首先要做的事是提供一个光标图像。过程如下：

1. 用 `wl_compositor` 创建一个新的 `wl_surface`。
2. 使用 `wl_pointer.set_cursor` 把该表面附加到指针上。
3. 给该表面附加一个光标图像 `wl_buffer` 并提交它。

这里唯一新引入的 API 是 `wl_pointer.set_cursor`：

```
<request name="set_cursor">
  <arg name="serial" type="uint" />
  <arg name="surface" type="object" interface="wl_surface" allow-
null="true" />
  <arg name="hotspot_x" type="int" />
  <arg name="hotspot_y" type="int" />
</request>
```

这里的 `serial` 必须来自 `enter` 事件。`hotspot_x` 和 `hotspot_y` 参数指定“热点”在光标表面局部坐标系中的位置，也就是指针在光标图像内的有效位置（例如箭头尖端处）。另请注意，表面可以为 `null`——用它来完全隐藏光标。

如果你正在寻找好的光标图像来源，`libwayland` 附带了一个独立的 `wayland-cursor` 库，它可以从磁盘加载 X 光标主题并为它们创建 `wl_buffers`。详情见 `wayland-cursor.h`，或参见我们在第 9.5 节中对示例客户端的更新。

- 注意：`wayland-cursor` 包含了处理动态光标的代码，而动态光标在 1998 年就已经不酷了。我要是你，才不会去碰它。从来没有人抱怨过我的 Wayland 客户端不支持动态光标。*

在光标进入你的表面且你已附加了合适的光标之后，你就可以开始处理输入事件了。有 `motion`、`button` 和 `axis` 三类事件。

指针帧

服务端上单帧输入处理可能携带大量变化的信息——例如，轮询一次鼠标可能在一个数据包中返回更新后的位置和某个按键的释放。服务端把这些变化作为单独的 *Wayland* 事件发送，并用“frame”事件把它们归为一组。

```
<event name="frame"></event>
```

客户端应当在其收到 `wl_pointer` 事件时把它们全部累积起来，然后在收到“frame”事件后，将待处理的输入作为单个指针事件来处理。

移动事件

移动事件所用的坐标空间与 `enter` 事件相同，并且相当直接：

```
<event name="motion">
  <arg name="time" type="uint" />
  <arg name="surface_x" type="fixed" />
  <arg name="surface_y" type="fixed" />
</event>
```

像所有带时间戳的输入事件一样，`time` 值是与该输入事件关联的、单调递增的毫秒级精度时间戳。

按键事件

按键事件基本不言自明：

```
<enum name="button_state">
  <entry name="released" value="0" />
  <entry name="pressed" value="1" />
</enum>

<event name="button">
  <arg name="serial" type="uint" />
  <arg name="time" type="uint" />
  <arg name="button" type="uint" />
  <arg name="state" type="uint" enum="button_state" />
</event>
```

不过，button 参数值得额外解释一下。这个数字是一个平台相关的输入事件编码，不过要注意 FreeBSD 复用了 Linux 的取值。你可以在 `linux/input-event-codes.h` 中找到 Linux 的这些取值，其中最有用的大概是常量 `BTN_LEFT`、`BTN_RIGHT` 和 `BTN_MIDDLE`。还有更多，就留给你闲暇时自行翻阅那个头文件吧。

轴事件

轴事件用于滚动操作，比如转动滚轮或把它从左向右摇摆。最基本的形式如下：

```
<enum name="axis">
  <entry name="vertical_scroll" value="0" />
  <entry name="horizontal_scroll" value="1" />
</enum>

<event name="axis">
  <arg name="time" type="uint" />
  <arg name="axis" type="uint" enum="axis" />
  <arg name="value" type="fixed" />
</event>
```

然而，轴事件很复杂，这也是多年来 `wl_pointer` 接口中最受关注的部分。存在若干额外事件来提高轴事件的针对性：

```

<enum name="axis_source">
  <entry name="wheel" value="0" />
  <entry name="finger" value="1" />
  <entry name="continuous" value="2" />
  <entry name="wheel_tilt" value="3" />
</enum>

<event name="axis_source" since="5">
  <arg name="axis_source" type="uint" enum="axis_source" />
</event>

```

axis_source 事件告诉你被触发的是哪种轴——滚轮、还是触摸板上的手指、把摇摆键拨向一侧、或者某种更新奇的东西。这个事件很简单，但其余的就没那么简单了：

```

<event name="axis_stop" since="5">
  <arg name="time" type="uint" />
  <arg name="axis" type="uint" enum="axis" />
</event>

<event name="axis_discrete" since="5">
  <arg name="axis" type="uint" enum="axis" />
  <arg name="discrete" type="int" />
</event>

```

这两个事件的确切语义很复杂，如果你想利用它们，我建议仔细阅读 wayland.xml 中的概述。简而言之，axis_discrete 事件用于在任意尺度的轴事件与离散步进（例如滚轮的每一次“咔哒”代表轴值的一次离散变化）之间加以区分。axis_stop 事件表示一次离散的用户动作已经完成，用于处理发生在若干帧内的一次滚动事件。此后任何事件都应被解读为一次独立的动作。

XKB 简介

我们清单上的下一个输入设备是键盘，但在讨论它们之前，我们需要先停下来给你补充一些背景知识。键盘映射是键盘输入中一个至关重要的细节，而 XKB 是在 Wayland 上处理它们的推荐方式。

当你在键盘上按下一个键时，它会向计算机发送一个 扫描码 (scancode)，这不过是分配给该物理按键的一个数字。在我的键盘上，扫描码 1 是 Escape 键，‘1’ 键是扫描码 2，‘a’ 是 30，Shift 是 42，依此类推。我用的是美式 ANSI 键盘布局，但还有许多其他布局，它们的扫描码各不相同。在我朋友的德式键盘上，扫描码 12 产生 ‘ß’，而我的产生 ‘.’。

为了解决这个问题，我们使用一个叫做 “xkbcommon” 的库，它的名字源于它所扮演的角色：从 XKB (X KeyBoard) 中提取出来、放入独立库的公共代码。XKB 定义了大量的键符号 (symbol)，例如 XKB_KEY_A、XKB_KEY_ssharp (ß，来自德语) 以及 XKB_KEY_kana_WO (を，来自日语)。

不过，识别这些键并把它们与这样的键符号对应起来，只是问题的一部分。按住 Shift 键时 ‘a’ 可以产生 ‘A’，在片假名模式下 ‘を’ 写作 ‘ワ’，而严格说来 ‘ß’ 虽然确实有大写形式，却几乎从不使用，更肯定从来不会有人去输入。像 Shift 这样的键叫做 修饰键 (modifier)，而像平假名、片假名这样的组叫做 组 (group)。有些修饰键可以 暂锁 (latch)，比如 Caps Lock。XKB 具备处理所有这些情况的原语，并维护一个状态机，用来跟踪你的键盘正在做什么，并准确判断出用户想要输入的是哪些 *Unicode* 码点。

使用 XKB

那么 xkbcommon 究竟是怎么用的呢？第一步是链接到它并引入头文件 `xkbcommon/xkbcommon.h`。¹ 大多数使用 xkbcommon 的程序都必须管理三个对象：

- `xkb_context`：用于配置其他 XKB 资源的句柄
- `xkb_keymap`：从扫描码到键符号的映射
- `xkb_state`：把键符号转换为 UTF-8 字符串的状态机

配置它的过程通常如下：

1. 使用 `xkb_context_new` 创建一个新的 `xkb_context`，除非你在做一些奇怪的事情，否则给它传 `XKB_CONTEXT_NO_FLAGS`。
2. 以字符串形式获取一个键盘映射。*

3. 使用 `xkb_keymap_new_from_string` 为该键盘映射创建一个 `xkb_keymap`。键盘映射只有一种格式，即 `XKB_KEYMAP_FORMAT_TEXT_V1`，你将把它传给 `format` 参数。同样，除非你在做一些奇怪的事情，否则 `flags` 就用 `XKB_KEYMAP_COMPILE_NO_FLAGS`。
4. 使用 `xkb_state_new` 配合你的键盘映射创建一个 `xkb_state`。该状态会增加键盘映射的引用计数，所以如果你自己不再需要它了，就用 `xkb_keymap_unref`。
5. 从键盘获取扫描码。*
6. 把这些扫描码喂给 `xkb_state_key_get_one_sym` 以得到键符号，喂给 `xkb_state_key_get_utf8` 以得到 UTF-8 字符串。嗒哒！

** 这些步骤将在下一节讨论。 *

从代码上说，这个过程如下所示：

```
#include <xkbcommon/xkbcommon.h> // -lxkbcommon
/* ... */

const char *keymap_str = /* ... */;

/* Create an XKB context */
struct xkb_context *context = xkb_context_new(XKB_CONTEXT_NO_FLAGS);

/* Use it to parse a keymap string */
struct xkb_keymap *keymap = xkb_keymap_new_from_string(
    xkb_context, keymap_str, XKB_KEYMAP_FORMAT_TEXT_V1,
    XKB_KEYMAP_COMPILE_NO_FLAGS);

/* Create an XKB state machine */
struct xkb_state *state = xkb_state_new(keymap);
```

然后，要处理扫描码：

```
int scancode = /* ... */;

xkb_keysym_t sym = xkb_state_key_get_one_sym(xkb_state, scancode);
if (sym == XKB_KEY_F1) {
    /* Do the thing you do when the user presses F1 */
}

char buf[128];
xkb_state_key_get_utf8(xkb_state, scancode, buf, sizeof(buf));
printf("UTF-8 input: %s\n", buf);
```

有了这些细节，我们就准备好着手处理键盘输入了。

1. `xkbcommon` 附带一个 `pc` 文件：使用 `pkgconf --cflags xkbcommon` 和 `pkgconf --libs xkbcommon`，或者你的构建系统首选的消费 `pc` 文件的方式。↩

键盘输入

有了对如何使用 XKB 的理解，让我们扩展 Wayland 代码，让它为我们提供可供其处理的按键事件。与获取 `wl_pointer` 资源的方式类似，我们可以使用 `wl_seat.get_keyboard` 请求，为一个能力包含 `WL_SEAT_CAPABILITY_KEYBOARD` 的 seat 创建 `wl_keyboard`。用完之后，你应该发送 “release” 请求：

```
<request name="release" type="destructor" since="3">
</request>
```

这会允许服务端清理与该键盘相关联的资源。

但究竟该怎么使用它呢？让我们从基础开始。

键盘映射

当你绑定到 `wl_keyboard` 时，服务端很可能发送的第一个事件就是 `keymap`。

```
<enum name="keymap_format">
  <entry name="no_keymap" value="0" />
  <entry name="xkb_v1" value="1" />
</enum>

<event name="keymap">
  <arg name="format" type="uint" enum="keymap_format" />
  <arg name="fd" type="fd" />
  <arg name="size" type="uint" />
</event>
```

提供 `keymap_format` 枚举，是为了以防我们将来为键盘映射想出某种新的格式，但在撰写本文时，XKB 键盘映射是服务端唯一可能发送的格式。

像这样的批量数据通过文件描述符传输。我们本可以简单地读取该文件描述符，但一般而言推荐改用 `mmap` 映射它。在 C 中，这看起来可能类似以下代码：

```

#include <sys/mman.h>
// ...

static void wl_keyboard_keymap(void *data, struct wl_keyboard
*wl_keyboard,
    uint32_t format, int32_t fd, uint32_t size) {
    assert(format == WL_KEYBOARD_KEYMAP_FORMAT_XKB_V1);
    struct my_state *state = (struct my_state *)data;

    char *map_shm = mmap(NULL, size, PROT_READ, MAP_PRIVATE, fd, 0);
    assert(map_shm != MAP_FAILED);

    struct xkb_keymap *keymap = xkb_keymap_new_from_string(
        state->xkb_context, map_shm, XKB_KEYMAP_FORMAT_TEXT_V1,
        XKB_KEYMAP_COMPILE_NO_FLAGS);
    munmap(map_shm, size);
    close(fd);

    // ...do something with keymap...
}

```

一旦我们有了键盘映射，就可以为这个 `wl_keyboard` 解释后续的按键事件了。注意服务端可以在任何时候发送新的键盘映射，所有后续的按键事件都应当据此来解释。

键盘焦点

```

<event name="enter">
    <arg name="serial" type="uint" />
    <arg name="surface" type="object" interface="wl_surface" />
    <arg name="keys" type="array" />
</event>

<event name="leave">
    <arg name="serial" type="uint" />
    <arg name="surface" type="object" interface="wl_surface" />
</event>

```

如同 `wl_pointer` 的 “enter” 和 “leave” 事件在指针移动到你的表面上时发出一样，当某个表面获得键盘焦点时服务端发送 `wl_keyboard.enter`，当它失去焦点时发送 `wl_keyboard.leave`。许多应用会在这些情况下改变外观——例如，开始绘制一个闪烁的光标。

“enter” 事件还包含一个当前按下的键的数组。这是一个 32 位无符号整数数组，每个元素代表一个被按下按键的扫描码。

输入事件

一旦键盘进入了你的表面，你就可以期待开始接收输入事件。

```
<enum name="key_state">
  <entry name="released" value="0" />
  <entry name="pressed" value="1" />
</enum>

<event name="key">
  <arg name="serial" type="uint" />
  <arg name="time" type="uint" />
  <arg name="key" type="uint" />
  <arg name="state" type="uint" enum="key_state" />
</event>

<event name="modifiers">
  <arg name="serial" type="uint" />
  <arg name="mods_depressed" type="uint" />
  <arg name="mods_latched" type="uint" />
  <arg name="mods_locked" type="uint" />
  <arg name="group" type="uint" />
</event>
```

“key”事件在用户按下或释放某个键时发送。像许多输入事件一样，它包含一个 serial，你可以用它把后续请求与这个输入事件关联起来。“key”是被按下或释放的那个键的扫描码，而“state”是该键的按下或释放状态。

重要：该事件中的扫描码是 Linux evdev 扫描码。要把它转换成 XKB 扫描码，你必须给 evdev 扫描码加上 8。

修饰键事件包含一个类似的 serial，以及按下、暂锁和锁定修饰键的掩码，还有当前使用的输入组的索引。例如，当你按住 Shift 时，某个修饰键就处于按下状态。修饰键可以暂锁，比如在启用粘滞键的情况下按下 Shift——在下次按下非修饰键之后它就会停止生效。而修饰键也可以锁定，比如当大写锁定被打开或关闭时。输入组用于在各种键盘布局之间切换，例如在 ISO 与 ANSI 布局之间切换，或用于更多与语言相关的特性。

修饰键的解释是依键盘映射而定的。你应当把它们两者都转发给 XKB 去处理。“modifiers”事件的大多数实现都很直接：

```
static void wl_keyboard_modifiers(void *data, struct wl_keyboard
*wl_keyboard,
    uint32_t serial, uint32_t depressed, uint32_t latched,
    uint32_t locked, uint32_t group) {
    struct my_state *state = (struct my_state *)data;
    xkb_state_update_mask(state->xkb_state,
        depressed, latched, locked, 0, 0, group);
}
```

按键重复

最后一个要考虑的事件是 “repeat_info” 事件：

```
<event name="repeat_info" since="4">
    <arg name="rate" type="int" />
    <arg name="delay" type="int" />
</event>
```

在 Wayland 中，客户端负责实现 “按键重复”——即只要你一直按住某个键不放，它就会持续输入字符的功能。这个事件用于告知客户端用户在按键重复设置方面的偏好。“delay” 是一个键在被按住多少毫秒之后按键重复才会启动，而 “rate” 则是直到该键被释放之前每秒重复的字符数。

触摸输入

从表面上看，触摸屏输入相当简单，你的实现也可以做得很简单。然而，该协议为你提供很大的深度，应用程序可以利用这些深度来提供更细致入微的触摸手势和反馈。

大多数触摸屏设备都支持多点触控（multitouch）：它们能够追踪屏幕上被触摸的多个位置。每一个这样的“触摸点”都会被分配一个 ID，该 ID 在当前所有处于活跃状态的屏幕触摸点中是唯一的，但如果你抬起手指再按下去，它可能会被复用。¹

与其他输入设备类似，你可以通过 `wl_seat.get_touch` 获取一个 `wl_touch` 资源，并在用完它之后发送一个“release”请求。

触摸帧

与指针一样，服务端上一帧（frame）的触摸处理可能携带许多变更的信息，但服务端会将这些变更作为离散的 Wayland 事件发送。`wl_touch.frame` 事件用于将这些事件归为一组。

```
<event name="frame"></event>
```

客户端应当在收到 `wl_touch` 事件时就累积它们，然后在收到“frame”事件时，把待处理的输入作为一个单一的触摸事件来处理。

按下与抬起

我们要看的首批事件是“down”和“up”，它们分别在你将手指按在设备上、以及将手指从设备上移开时被触发。

```

<event name="down">
  <arg name="serial" type="uint" />
  <arg name="time" type="uint" />
  <arg name="surface" type="object" interface="wl_surface" />
  <arg name="id" type="int" />
  <arg name="x" type="fixed" />
  <arg name="y" type="fixed" />
</event>

<event name="up">
  <arg name="serial" type="uint" />
  <arg name="time" type="uint" />
  <arg name="id" type="int" />
</event>

```

“x”和“y”坐标是定点（fixed-point）坐标，位于被触摸的表面的坐标空间中——该表面由“surface”参数给出。时间是单调递增的时间戳，其纪元（epoch）是任意的，单位为毫秒。² 另请注意其中包含了一个 serial，它可以被包含在后续的请求中，以便将这些请求与该输入事件关联起来。

移动

在你收到带有特定触摸 ID 的“down”事件之后，你将开始收到描述该触摸点在设备上移动情况的移动事件。

```

<event name="motion">
  <arg name="time" type="uint" />
  <arg name="id" type="int" />
  <arg name="x" type="fixed" />
  <arg name="y" type="fixed" />
</event>

```

这里的“x”和“y”坐标位于“enter”事件所针对的那个表面的相对坐标空间中。

手势取消

触摸事件往往必须达到某个阈值，才会被识别为一个手势。例如，在屏幕上从左向右滑动，可能被 Wayland 合成器用来在应用程序之间切换。然而，只有当跨越了某个阈值之后——比如说，在一定时间内到达屏幕的中点——合成器才会将该行为识别为一个手势。

在达到该阈值之前，合成器会与被触摸的表面发送正常的触摸事件。一旦识别出手势，合成器就会发送一个“cancel”事件，让你知道合成器正在接管。

```
<event name="cancel"></event>
```

当你收到该事件时，所有活跃的触摸点都被取消。

形状与方向

一些高端触摸硬件能够确定更多关于用户与其交互方式的信息。对于使用合适硬件的用户，以及希望采用更高级的交互或触摸反馈的应用程序，协议提供了“shape”和“orientation”事件。

```
<event name="shape" since="6">
  <arg name="id" type="int" />
  <arg name="major" type="fixed" />
  <arg name="minor" type="fixed" />
</event>

<event name="orientation" since="6">
  <arg name="id" type="int" />
  <arg name="orientation" type="fixed" />
</event>
```

“shape”事件定义了一个对正在触摸屏幕的物体形状的椭圆近似，其长轴和短轴以被触摸表面的坐标空间中的单位来表示。orientation 事件则通过指定长轴与被触摸表面的 Y 轴之间的夹角（以度为单位）来旋转这个椭圆。

触摸是 Wayland 协议所支持的最后一类输入设备。掌握了这些知识，让我们来更新我们的示例代码。

1. 重点在于“可能”——不要基于触摸点 ID 的重复使用做出任何假设。↩
2. 这意味着，各个时间戳之间可以相互比较，从而得出事件之间的时间间隔，但它们无法与墙上时钟（wall-clock）时间相比较。↩

扩展示例代码

在前面的章节中，我们构建了一个简单的客户端，它能在显示器上呈现自己的表面。让我们稍微扩展这段代码，构建一个能够接收输入事件的客户端。为了简单起见，我们只把输入事件记录到 stderr。

这需要的代码量比我们目前接触过的要多得多，所以请系好安全带。我们首先需要做的是建立 seat。

建立 seat

我们需要的第一样东西是一个指向 seat 的引用。我们会把它加到 client_state 结构中，同时加上键盘、指针和触摸对象，供稍后使用：

```
struct wl_shm *wl_shm;
struct wl_compositor *wl_compositor;
struct xdg_wm_base *xdg_wm_base;
+ struct wl_seat *wl_seat;
  /* Objects */
  struct wl_surface *wl_surface;
  struct xdg_surface *xdg_surface;
+ struct wl_keyboard *wl_keyboard;
+ struct wl_pointer *wl_pointer;
+ struct wl_touch *wl_touch;
  /* State */
  float offset;
  uint32_t last_frame;
  int width, height;
```

我们还需要更新 registry_global，为该 seat 注册一个监听器。

```
                                wl_registry, name,
&xdg_wm_base_interface, 1);
                                xdg_wm_base_add_listener(state->xdg_wm_base,
                                                                &xdg_wm_base_listener, state);
+     } else if (strcmp(interface, wl_seat_interface.name) == 0) {
+         state->wl_seat = wl_registry_bind(
+             wl_registry, name, &wl_seat_interface,
7);
+         wl_seat_add_listener(state->wl_seat,
+                               &wl_seat_listener, state);
+     }
}
```

注意，我们绑定的是 seat 接口的最新版本，即版本 7。让我们也把那个监听器搭起来：

```
+static void
+wl_seat_capabilities(void *data, struct wl_seat *wl_seat, uint32_t
+capabilities)
+{
+    struct client_state *state = data;
+    /* TODO */
+}
+
+static void
+wl_seat_name(void *data, struct wl_seat *wl_seat, const char *name)
+{
+    fprintf(stderr, "seat name: %s\n", name);
+}
+
+static const struct wl_seat_listener wl_seat_listener = {
+    .capabilities = wl_seat_capabilities,
+    .name = wl_seat_name,
+};
```

如果你现在编译（`cc -o client client.c xdg-shell-protocol.c`）并运行，应该能看到 seat 的名字被打印到 stderr。

搭好指针事件

让我们来看看指针事件。如果你还记得，来自 Wayland 服务端的指针事件要被累积成一个逻辑事件。因此，我们需要定义一个结构体来存储它们。

```

+enum pointer_event_mask {
+    POINTER_EVENT_ENTER = 1 << 0,
+    POINTER_EVENT_LEAVE = 1 << 1,
+    POINTER_EVENT_MOTION = 1 << 2,
+    POINTER_EVENT_BUTTON = 1 << 3,
+    POINTER_EVENT_AXIS = 1 << 4,
+    POINTER_EVENT_AXIS_SOURCE = 1 << 5,
+    POINTER_EVENT_AXIS_STOP = 1 << 6,
+    POINTER_EVENT_AXIS_DISCRETE = 1 << 7,
+};
+
+struct pointer_event {
+    uint32_t event_mask;
+    wl_fixed_t surface_x, surface_y;
+    uint32_t button, state;
+    uint32_t time;
+    uint32_t serial;
+    struct {
+        bool valid;
+        wl_fixed_t value;
+        int32_t discrete;
+    } axes[2];
+    uint32_t axis_source;
+};

```

这里我们会用一个位掩码来标识单个指针帧中接收到了哪些事件，并把每个事件的相关信息存储到各自的字段中。让我们也把它加到我们的状态结构体里：

```

    /* State */
    float offset;
    uint32_t last_frame;
    int width, height;
    bool closed;
+    struct pointer_event pointer_event;
};

```

接下来我们需要更新 `wl_seat_capabilities`，为那些支持指针输入的 `seat` 建立指针对象。

```

static void
wl_seat_capabilities(void *data, struct wl_seat *wl_seat, uint32_t
capabilities)
{
    struct client_state *state = data;
-    /* TODO */
+
+    bool have_pointer = capabilities & WL_SEAT_CAPABILITY_POINTER;
+
+    if (have_pointer && state->wl_pointer == NULL) {
+        state->wl_pointer = wl_seat_get_pointer(state-
>wl_seat);
+        wl_pointer_add_listener(state->wl_pointer,
+                                &wl_pointer_listener, state);
+    } else if (!have_pointer && state->wl_pointer != NULL) {
+        wl_pointer_release(state->wl_pointer);
+        state->wl_pointer = NULL;
+    }
}

```

这里值得稍作解释。回想一下，capabilities 是一个位掩码，表示该 seat 支持哪些类型的设备——用它与某个能力做按位与（&），如果支持，结果就非零。然后，如果我们有指针，并且尚未配置过它，我们就走第一个分支，用 wl_seat_get_pointer 获取一个指针引用并存入我们的状态中。如果该 seat 不支持指针，但我们已经配置了一个，我们就用 wl_pointer_release 把它释放掉。记住，seat 的能力可能在运行期发生变化，比如用户拔掉又插上他们的鼠标时。

我们还为指针配置了一个监听器。让我们也把它结构体加上：

```

+static const struct wl_pointer_listener wl_pointer_listener = {
+    .enter = wl_pointer_enter,
+    .leave = wl_pointer_leave,
+    .motion = wl_pointer_motion,
+    .button = wl_pointer_button,
+    .axis = wl_pointer_axis,
+    .frame = wl_pointer_frame,
+    .axis_source = wl_pointer_axis_source,
+    .axis_stop = wl_pointer_axis_stop,
+    .axis_discrete = wl_pointer_axis_discrete,
+};

```

指针有很多事件。让我们来看看它们。

```

+static void
+wl_pointer_enter(void *data, struct wl_pointer *wl_pointer,
+                uint32_t serial, struct wl_surface *surface,
+                wl_fixed_t surface_x, wl_fixed_t surface_y)
+{
+    struct client_state *client_state = data;
+    client_state->pointer_event.event_mask |= POINTER_EVENT_ENTER;
+    client_state->pointer_event.serial = serial;
+    client_state->pointer_event.surface_x = surface_x,
+        client_state->pointer_event.surface_y = surface_y;
+}
+
+static void
+wl_pointer_leave(void *data, struct wl_pointer *wl_pointer,
+                uint32_t serial, struct wl_surface *surface)
+{
+    struct client_state *client_state = data;
+    client_state->pointer_event.serial = serial;
+    client_state->pointer_event.event_mask |= POINTER_EVENT_LEAVE;
+}

```

“enter”和“leave”事件相当直接，它们为实现的其余部分奠定了基础。我们把相应的事件加进事件掩码，然后用提供给我们的数据填充它。“motion”和“button”事件则颇为类似：

```

+static void
+wl_pointer_motion(void *data, struct wl_pointer *wl_pointer, uint32_t
time,
+                wl_fixed_t surface_x, wl_fixed_t surface_y)
+{
+    struct client_state *client_state = data;
+    client_state->pointer_event.event_mask |=
POINTER_EVENT_MOTION;
+    client_state->pointer_event.time = time;
+    client_state->pointer_event.surface_x = surface_x,
+        client_state->pointer_event.surface_y = surface_y;
+}
+
+static void
+wl_pointer_button(void *data, struct wl_pointer *wl_pointer, uint32_t
serial,
+                uint32_t time, uint32_t button, uint32_t state)
+{
+    struct client_state *client_state = data;
+    client_state->pointer_event.event_mask |=
POINTER_EVENT_BUTTON;
+    client_state->pointer_event.time = time;
+    client_state->pointer_event.serial = serial;
+    client_state->pointer_event.button = button,
+        client_state->pointer_event.state = state;
+}

```

轴 (axis) 事件要稍微复杂一些, 因为有两个轴: 水平和垂直。因此, 我们的 `pointer_event` 结构体中包含一个数组, 里面有两组轴事件。处理它们我们的代码最后会写成这样:

```
+static void
+wl_pointer_axis(void *data, struct wl_pointer *wl_pointer, uint32_t
+time,
+                uint32_t axis, wl_fixed_t value)
+{
+    struct client_state *client_state = data;
+    client_state->pointer_event.event_mask |= POINTER_EVENT_AXIS;
+    client_state->pointer_event.time = time;
+    client_state->pointer_event.axes[axis].valid = true;
+    client_state->pointer_event.axes[axis].value = value;
+}
+
+static void
+wl_pointer_axis_source(void *data, struct wl_pointer *wl_pointer,
+                        uint32_t axis_source)
+{
+    struct client_state *client_state = data;
+    client_state->pointer_event.event_mask |=
+POINTER_EVENT_AXIS_SOURCE;
+    client_state->pointer_event.axis_source = axis_source;
+}
+
+static void
+wl_pointer_axis_stop(void *data, struct wl_pointer *wl_pointer,
+                      uint32_t time, uint32_t axis)
+{
+    struct client_state *client_state = data;
+    client_state->pointer_event.time = time;
+    client_state->pointer_event.event_mask |=
+POINTER_EVENT_AXIS_STOP;
+    client_state->pointer_event.axes[axis].valid = true;
+}
+
+static void
+wl_pointer_axis_discrete(void *data, struct wl_pointer *wl_pointer,
+                           uint32_t axis, int32_t discrete)
+{
+    struct client_state *client_state = data;
+    client_state->pointer_event.event_mask |=
+POINTER_EVENT_AXIS_DISCRETE;
+    client_state->pointer_event.axes[axis].valid = true;
+    client_state->pointer_event.axes[axis].discrete = discrete;
+}
```

同样相当直接, 除了主要的变化——更新受影响的那个轴。另外注意“valid”布尔值的用法: 我们有可能收到一个指针帧只更新了某一个轴, 而没有更新另一个, 所以我们用这个“valid”值来判断在帧事件中哪些轴被更新了。

说到这个，该是我们的重头戏了：我们的“frame”处理函数。

```

+static void
+wl_pointer_frame(void *data, struct wl_pointer *wl_pointer)
+{
+    struct client_state *client_state = data;
+    struct pointer_event *event = &client_state->pointer_event;
+    fprintf(stderr, "pointer frame @ %d: ", event->time);
+
+    if (event->event_mask & POINTER_EVENT_ENTER) {
+        fprintf(stderr, "entered %f, %f ",
+                wl_fixed_to_double(event->surface_x),
+                wl_fixed_to_double(event->surface_y));
+    }
+
+    if (event->event_mask & POINTER_EVENT_LEAVE) {
+        fprintf(stderr, "leave");
+    }
+
+    if (event->event_mask & POINTER_EVENT_MOTION) {
+        fprintf(stderr, "motion %f, %f ",
+                wl_fixed_to_double(event->surface_x),
+                wl_fixed_to_double(event->surface_y));
+    }
+
+    if (event->event_mask & POINTER_EVENT_BUTTON) {
+        char *state = event->state ==
WL_POINTER_BUTTON_STATE_RELEASED ?
+        "released" : "pressed";
+        fprintf(stderr, "button %d %s ", event->button,
state);
+    }
+
+    uint32_t axis_events = POINTER_EVENT_AXIS
+        | POINTER_EVENT_AXIS_SOURCE
+        | POINTER_EVENT_AXIS_STOP
+        | POINTER_EVENT_AXIS_DISCRETE;
+    char *axis_name[2] = {
+        [WL_POINTER_AXIS_VERTICAL_SCROLL] = "vertical",
+        [WL_POINTER_AXIS_HORIZONTAL_SCROLL] = "horizontal",
+    };
+    char *axis_source[4] = {
+        [WL_POINTER_AXIS_SOURCE_WHEEL] = "wheel",
+        [WL_POINTER_AXIS_SOURCE_FINGER] = "finger",
+        [WL_POINTER_AXIS_SOURCE_CONTINUOUS] = "continuous",
+        [WL_POINTER_AXIS_SOURCE_WHEEL_TILT] = "wheel tilt",
+    };
+    if (event->event_mask & axis_events) {
+        for (size_t i = 0; i < 2; ++i) {
+            if (!event->axes[i].valid) {
+                continue;
+            }
+            fprintf(stderr, "%s axis ", axis_name[i]);
+            if (event->event_mask & POINTER_EVENT_AXIS) {
+                fprintf(stderr, "value %f ",
wl_fixed_to_double(

```



```

@@ -1,4 +1,5 @@
#define _POSIX_C_SOURCE 200112L
#include <assert.h>
#include <errno.h>
#include <fcntl.h>
#include <limits.h>
@@ -9,6 +10,7 @@
#include <time.h>
#include <unistd.h>
#include <wayland-client.h>
#include <xkbcommon/xkbcommon.h>
#include "xdg-shell-client-protocol.h"

```

我们还需要在 main 函数中初始化 xkb_context:

```

@@ -603,6 +649,7 @@ main(int argc, char *argv[])
    state.height = 480;
    state.wl_display = wl_display_connect(NULL);
    state.wl_registry = wl_display_get_registry(state.wl_display);
+   state.xkb_context = xkb_context_new(XKB_CONTEXT_NO_FLAGS);
    wl_registry_add_listener(state.wl_registry,
&wl_registry_listener, &state);
    wl_display_roundtrip(state.wl_display);

```

接下来, 让我们更新 seat 能力函数, 把键盘监听器也搭起来。

```

        } else if (!have_pointer && state->wl_pointer != NULL) {
            wl_pointer_release(state->wl_pointer);
            state->wl_pointer = NULL;
        }
+
+   bool have_keyboard = capabilities &
WL_SEAT_CAPABILITY_KEYBOARD;
+
+   if (have_keyboard && state->wl_keyboard == NULL) {
+       state->wl_keyboard = wl_seat_get_keyboard(state-
>wl_seat);
+       wl_keyboard_add_listener(state->wl_keyboard,
+                               &wl_keyboard_listener, state);
+   } else if (!have_keyboard && state->wl_keyboard != NULL) {
+       wl_keyboard_release(state->wl_keyboard);
+       state->wl_keyboard = NULL;
+   }
}

```

我们还得定义这里用到的 wl_keyboard_listener。

```
+static const struct wl_keyboard_listener wl_keyboard_listener = {
+    .keymap = wl_keyboard_keymap,
+    .enter = wl_keyboard_enter,
+    .leave = wl_keyboard_leave,
+    .key = wl_keyboard_key,
+    .modifiers = wl_keyboard_modifiers,
+    .repeat_info = wl_keyboard_repeat_info,
+};
```

现在到了改动的重头部分。让我们从键盘映射开始：

```
+static void
+wl_keyboard_keymap(void *data, struct wl_keyboard *wl_keyboard,
+    uint32_t format, int32_t fd, uint32_t size)
+{
+    struct client_state *client_state = data;
+    assert(format == WL_KEYBOARD_KEYMAP_FORMAT_XKB_V1);
+
+    char *map_shm = mmap(NULL, size, PROT_READ, MAP_SHARED, fd,
0);
+    assert(map_shm != MAP_FAILED);
+
+    struct xkb_keymap *xkb_keymap = xkb_keymap_new_from_string(
+        client_state->xkb_context, map_shm,
+        XKB_KEYMAP_FORMAT_TEXT_V1,
XKB_KEYMAP_COMPILE_NO_FLAGS);
+    munmap(map_shm, size);
+    close(fd);
+
+    struct xkb_state *xkb_state = xkb_state_new(xkb_keymap);
+    xkb_keymap_unref(client_state->xkb_keymap);
+    xkb_state_unref(client_state->xkb_state);
+    client_state->xkb_keymap = xkb_keymap;
+    client_state->xkb_state = xkb_state;
+}
```

现在我们可以看到为什么加了 `assert.h`——我们在这里用它来确保键盘映射的格式是我们所期望的那种。然后，我们用 `mmap` 把合成器发给我们的文件描述符映射到一个 `char *` 指针上，好把它传给 `xkb_keymap_new_from_string`。别忘了之后要 `munmap` 并 `close` 那个 `fd`——接着我们建立 XKB 状态。另外注意，我们还 `unref` 了此前在调用这个函数时设置好的任何 XKB 键盘映射或状态，以防合成器在运行期更换键盘映射。¹

```

+static void
+wl_keyboard_enter(void *data, struct wl_keyboard *wl_keyboard,
+                  uint32_t serial, struct wl_surface *surface,
+                  struct wl_array *keys)
+{
+    struct client_state *client_state = data;
+    fprintf(stderr, "keyboard enter; keys pressed are:\n");
+    uint32_t *key;
+    wl_array_for_each(key, keys) {
+        char buf[128];
+        xkb_keysym_t sym = xkb_state_key_get_one_sym(
+                           client_state->xkb_state, *key + 8);
+        xkb_keysym_get_name(sym, buf, sizeof(buf));
+        fprintf(stderr, "sym: %-12s (%d), ", buf, sym);
+        xkb_state_key_get_utf8(client_state->xkb_state,
+                                *key + 8, buf, sizeof(buf));
+        fprintf(stderr, "utf8: '%s'\n", buf);
+    }
+}

```

当键盘“进入”我们的表面时，我们就获得了键盘焦点。合成器会转发一份当时已经被按下的按键列表，这里我们只是把它们枚举出来，并记录它们的 keysym 名称和等价的 UTF-8 字符。当按键被按下时，我们会做类似的事情：

```

+static void
+wl_keyboard_key(void *data, struct wl_keyboard *wl_keyboard,
+                uint32_t serial, uint32_t time, uint32_t key, uint32_t
+                state)
+{
+    struct client_state *client_state = data;
+    char buf[128];
+    uint32_t keycode = key + 8;
+    xkb_keysym_t sym = xkb_state_key_get_one_sym(
+                       client_state->xkb_state, keycode);
+    xkb_keysym_get_name(sym, buf, sizeof(buf));
+    const char *action =
+        state == WL_KEYBOARD_KEY_STATE_PRESSED ? "press" :
+        "release";
+    fprintf(stderr, "key %s: sym: %-12s (%d), ", action, buf,
+    sym);
+    xkb_state_key_get_utf8(client_state->xkb_state, keycode,
+                            buf, sizeof(buf));
+    fprintf(stderr, "utf8: '%s'\n", buf);
+}

```

最后，我们为剩下的三个事件加上简单的实现：

```

+static void
+wl_keyboard_leave(void *data, struct wl_keyboard *wl_keyboard,
+                  uint32_t serial, struct wl_surface *surface)
+{
+    fprintf(stderr, "keyboard leave\n");
+}
+
+static void
+wl_keyboard_modifiers(void *data, struct wl_keyboard *wl_keyboard,
+                      uint32_t serial, uint32_t mods_depressed,
+                      uint32_t mods_latched, uint32_t mods_locked,
+                      uint32_t group)
+{
+    struct client_state *client_state = data;
+    xkb_state_update_mask(client_state->xkb_state,
+                          mods_depressed, mods_latched, mods_locked, 0, 0,
+                          group);
+}
+
+static void
+wl_keyboard_repeat_info(void *data, struct wl_keyboard *wl_keyboard,
+                        int32_t rate, int32_t delay)
+{
+    /* Left as an exercise for the reader */
+}

```

对于修饰键，我们可以进一步解码它们，但大多数应用并不需要。我们在这里只是更新 XKB 状态。至于处理按键重复——这有很多因应用而异的约束条件。你想重复文本输入吗？你想重复键盘快捷键吗？它们的时序又如何与你的事件循环交互？这些问题的答案留给你自己决定。

如果你再次编译这段代码，你应该能在窗口中开始输入，并看到你的输入被打印到日志里。呼啦！

搭好触摸事件

最后，我们会为支持触摸的设备添加支持。和指针一样，触摸设备也存在“frame”事件。不过，它们因为可能在一个帧内更新多个触摸点而变得更加复杂。我们会再加一些结构体和枚举来表示累积的状态：

```

+enum touch_event_mask {
+    TOUCH_EVENT_DOWN = 1 << 0,
+    TOUCH_EVENT_UP = 1 << 1,
+    TOUCH_EVENT_MOTION = 1 << 2,
+    TOUCH_EVENT_CANCEL = 1 << 3,
+    TOUCH_EVENT_SHAPE = 1 << 4,
+    TOUCH_EVENT_ORIENTATION = 1 << 5,
+};
+
+struct touch_point {
+    bool valid;
+    int32_t id;
+    uint32_t event_mask;
+    wl_fixed_t surface_x, surface_y;
+    wl_fixed_t major, minor;
+    wl_fixed_t orientation;
+};
+
+struct touch_event {
+    uint32_t event_mask;
+    uint32_t time;
+    uint32_t serial;
+    struct touch_point points[10];
+};

```

注意，我在这里随意选了 10 个触摸点，前提是大多数用户最多也就用这么多手指。对于更大的、多用户的触摸屏，你可能需要一个更高的上限。此外，有些触摸硬件同时支持的触摸点少于 10 个——8 个也很常见，而在较老的设备中，支持更少触摸点的硬件更是屡见不鲜。

我们会把这个结构体加到 `client_state` 中：

```

@@ -110,6 +135,7 @@ struct client_state {
    struct xkb_state *xkb_state;
    struct xkb_context *xkb_context;
    struct xkb_keymap *xkb_keymap;
+    struct touch_event touch_event;
};

```

并且我们会更新 `seat` 能力处理器，在有触摸支持时搭起一个监听器。

```

    } else if (!have_keyboard && state->wl_keyboard != NULL) {
        wl_keyboard_release(state->wl_keyboard);
        state->wl_keyboard = NULL;
    }
+
+   bool have_touch = capabilities & WL_SEAT_CAPABILITY_TOUCH;
+
+   if (have_touch && state->wl_touch == NULL) {
+       state->wl_touch = wl_seat_get_touch(state->wl_seat);
+       wl_touch_add_listener(state->wl_touch,
+                             &wl_touch_listener, state);
+   } else if (!have_touch && state->wl_touch != NULL) {
+       wl_touch_release(state->wl_touch);
+       state->wl_touch = NULL;
+   }
+ }

```

我们又重复了一遍处理 seat 上触摸能力出现和消失的模式，所以我们对设备在运行期的出现和消失是稳健的。不过，触摸设备被热插拔的情况相对少见。

监听器本身在这里：

```

+static const struct wl_touch_listener wl_touch_listener = {
+   .down = wl_touch_down,
+   .up = wl_touch_up,
+   .motion = wl_touch_motion,
+   .frame = wl_touch_frame,
+   .cancel = wl_touch_cancel,
+   .shape = wl_touch_shape,
+   .orientation = wl_touch_orientation,
+};

```

为了处理多个触摸点，我们需要写一个小的辅助函数：

```

+static struct touch_point *
+get_touch_point(struct client_state *client_state, int32_t id)
+{
+    struct touch_event *touch = &client_state->touch_event;
+    const size_t nmemb = sizeof(touch->points) / sizeof(struct
touch_point);
+    int invalid = -1;
+    for (size_t i = 0; i < nmemb; ++i) {
+        if (touch->points[i].id == id) {
+            return &touch->points[i];
+        }
+        if (invalid == -1 && !touch->points[i].valid) {
+            invalid = i;
+        }
+    }
+    if (invalid == -1) {
+        return NULL;
+    }
+    touch->points[invalid].valid = true;
+    touch->points[invalid].id = id;
+    return &touch->points[invalid];
+}

```

这个函数的基本用途是：根据我们正在接收事件的触摸 ID，从我们加到 `touch_event` 结构体中的数组里挑出一个 `touch_point`。如果我们找到了该 ID 对应的既有 `touch_point`，就把它返回。如果没有，就返回第一个可用的触摸点。万一用完了，就返回 `NULL`。

现在我们可以利用这一点来实现我们的第一个函数：触摸按下（touch down）。

```

+static void
+wl_touch_down(void *data, struct wl_touch *wl_touch, uint32_t serial,
+    uint32_t time, struct wl_surface *surface, int32_t id,
+    wl_fixed_t x, wl_fixed_t y)
+{
+    struct client_state *client_state = data;
+    struct touch_point *point = get_touch_point(client_state, id);
+    if (point == NULL) {
+        return;
+    }
+    point->event_mask |= TOUCH_EVENT_UP;
+    point->surface_x = wl_fixed_to_double(x),
+        point->surface_y = wl_fixed_to_double(y);
+    client_state->touch_event.time = time;
+    client_state->touch_event.serial = serial;
+}

```

和指针事件一样，我们也只是把这个状态累积起来以便稍后使用。我们当时还不知道这个事件是否代表一个完整的触摸帧。让我们为触摸抬起（touch up）加上类似的东西：

```

+static void
+wl_touch_up(void *data, struct wl_touch *wl_touch, uint32_t serial,
+            uint32_t time, int32_t id)
+{
+    struct client_state *client_state = data;
+    struct touch_point *point = get_touch_point(client_state, id);
+    if (point == NULL) {
+        return;
+    }
+    point->event_mask |= TOUCH_EVENT_UP;
+}

```

以及移动:

```

+static void
+wl_touch_motion(void *data, struct wl_touch *wl_touch, uint32_t time,
+               int32_t id, wl_fixed_t x, wl_fixed_t y)
+{
+    struct client_state *client_state = data;
+    struct touch_point *point = get_touch_point(client_state, id);
+    if (point == NULL) {
+        return;
+    }
+    point->event_mask |= TOUCH_EVENT_MOTION;
+    point->surface_x = x, point->surface_y = y;
+    client_state->touch_event.time = time;
+}

```

触摸取消事件有些不同, 因为它会一次性“取消”所有活动的触摸点。我们就把它存到 touch_event 顶层的那个事件掩码里。

```

+static void
+wl_touch_cancel(void *data, struct wl_touch *wl_touch)
+{
+    struct client_state *client_state = data;
+    client_state->touch_event.event_mask |= TOUCH_EVENT_CANCEL;
+}

```

形状 (shape) 和方向 (orientation) 事件与按下、抬起、移动类似, 只不过它们告知我们某个特定触摸点的尺寸信息。

```

+static void
+wl_touch_shape(void *data, struct wl_touch *wl_touch,
+               int32_t id, wl_fixed_t major, wl_fixed_t minor)
+{
+    struct client_state *client_state = data;
+    struct touch_point *point = get_touch_point(client_state, id);
+    if (point == NULL) {
+        return;
+    }
+    point->event_mask |= TOUCH_EVENT_SHAPE;
+    point->major = major, point->minor = minor;
+}
+
+static void
+wl_touch_orientation(void *data, struct wl_touch *wl_touch,
+                     int32_t id, wl_fixed_t orientation)
+{
+    struct client_state *client_state = data;
+    struct touch_point *point = get_touch_point(client_state, id);
+    if (point == NULL) {
+        return;
+    }
+    point->event_mask |= TOUCH_EVENT_ORIENTATION;
+    point->orientation = orientation;
+}

```

最后，在收到一个帧事件时，我们可以把所有累积的这些状态解释为单个输入事件，就像我们的指针代码那样。

```

+static void
+wl_touch_frame(void *data, struct wl_touch *wl_touch)
+{
+    struct client_state *client_state = data;
+    struct touch_event *touch = &client_state->touch_event;
+    const size_t nmemb = sizeof(touch->points) / sizeof(struct
touch_point);
+    fprintf(stderr, "touch event @ %d:\n", touch->time);
+
+    for (size_t i = 0; i < nmemb; ++i) {
+        struct touch_point *point = &touch->points[i];
+        if (!point->valid) {
+            continue;
+        }
+        fprintf(stderr, "point %d: ", touch->points[i].id);
+
+        if (point->event_mask & TOUCH_EVENT_DOWN) {
+            fprintf(stderr, "down %f,%f ",
+                wl_fixed_to_double(point-
>surface_x),
+                wl_fixed_to_double(point-
>surface_y));
+        }
+
+        if (point->event_mask & TOUCH_EVENT_UP) {
+            fprintf(stderr, "up ");
+        }
+
+        if (point->event_mask & TOUCH_EVENT_MOTION) {
+            fprintf(stderr, "motion %f,%f ",
+                wl_fixed_to_double(point-
>surface_x),
+                wl_fixed_to_double(point-
>surface_y));
+        }
+
+        if (point->event_mask & TOUCH_EVENT_SHAPE) {
+            fprintf(stderr, "shape %fx%f ",
+                wl_fixed_to_double(point-
>major),
+                wl_fixed_to_double(point-
>minor));
+        }
+
+        if (point->event_mask & TOUCH_EVENT_ORIENTATION) {
+            fprintf(stderr, "orientation %f ",
+                wl_fixed_to_double(point-
>orientation));
+        }
+
+        point->valid = false;
+        fprintf(stderr, "\n");
+    }
+}

```

再次编译并运行，当你与触摸设备交互时，你就能看到触摸事件被打印到 `stderr`（前提是你有这样一台设备可供测试）。现在我们的客户端支持输入了！

接下来呢？

输入设备的种类繁多，所以扩展我们的代码来支持它们着实要费一番功夫——仅在这一章里，我们的代码量就增长了 2.5 倍。不过回报应该挺可观的，因为现在你已经熟悉了足够多的 Wayland 概念（和代码），足以实现很多客户端了。

还有一点点要学的——在最后几章里，我们会介绍弹出窗口、上下文菜单、交互式窗口移动与缩放、剪贴板以及拖放支持，稍后还会介绍一些支持更小众用例的有趣的协议扩展。我强烈建议你在开始构建自己的客户端之前，至少读一读第 10.1 节，因为它涵盖了诸如在合成器请求时调整窗口大小之类的内容。

-
1. 这在实践中确实会发生！ [↩](#)

深入 XDG shell

到目前为止，我们已经能够在屏幕上把一个应用显示在顶层窗口中了，但 XDG shell 还有更多内容尚未被我们充分了解。即便是最简单的应用，正确地实现配置

（configuration）生命周期也会大有裨益，而且 xdg-shell 也为更复杂的应用提供了实用的特性。xdg-shell 的完整特性集涵盖：客户端与服务端就窗口尺寸进行的协商、多窗口层级结构、客户端侧装饰（client-side decorations），以及诸如上下文菜单这类窗口的语义化定位。

配置与生命周期

此前，我们创建了一个尺寸由我们自己选定的窗口：640x480。然而，合成器往往对窗口应当采用什么尺寸有自己的看法，而我们可能也想传达自己的偏好。若不能妥善处理这一点，通常会导致不理想的行为，比如合成器正试图告诉你把表面调小一些，结果却把你的窗口部分裁掉了。

合成器还可以向应用提供关于其展示上下文的额外线索。它可以让你知道应用是被最大化了还是处于全屏、是贴靠在其他窗口或显示器边缘的一侧或多侧、是获得焦点还是空闲，等等。由于 `wl_surface` 被用于以原子方式将表面的变更从客户端传达到服务端，`xdg_surface` 接口提供了下面这两条消息，用于让合成器提出变更建议、让客户端确认这些变更：

```
<request name="ack_configure">
  <arg name="serial" type="uint" />
</request>

<event name="configure">
  <arg name="serial" type="uint" />
</event>
```

单看这两条消息，它们几乎不携带什么含义。不过，`xdg_surface` 的每个子类（`xdg_toplevel` 和 `xdg_popup`）都有额外的事件，服务端可以在“configure”之前发送这些事件，以落实我们前面提到的每一项建议。服务端会把这些状态全部发送出去：已最大化、已获得焦点、建议的尺寸；然后再发送一个带有 `serial` 的 `configure` 事件。当客户端采用了与这些建议一致的状态后，它会发送一个带有相同 `serial` 的 `ack_configure` 请求来表示这一点。在随后对相关联的 `wl_surface` 进行一次提交之后，合成器便会认为该状态是一致的。

XDG 顶层窗口生命周期

我们第 7 章里的示例代码可以正常工作，但它目前还不是桌面上的良好公民。它不会采用合成器建议的尺寸，而且如果用户尝试关闭窗口，它也不会消失。要响应这些由合成器提供的事件，涉及两个 Wayland 事件：`configure` 和 `close`。

```
<event name="configure">
  <arg name="width" type="int"/>
  <arg name="height" type="int"/>
  <arg name="states" type="array"/>
</event>

<event name="close" />
```

width 和 height 是合成器为该窗口首选的尺寸¹，而 states 是下列取值构成的数组：

```
<enum name="state">
  <entry name="maximized" />
  <entry name="fullscreen" />
  <entry name="resizing" />
  <entry name="activated" />
  <entry name="tiled_left" />
  <entry name="tiled_right" />
  <entry name="tiled_top" />
  <entry name="tiled_bottom" />
</enum>
```

close 事件可以忽略，一个典型的原因是向用户显示一个确认，以便保存尚未保存的工作。我们第 7 章的示例代码可以相当轻松地更新以支持这些事件：

```

diff --git a/client.c b/client.c
--- a/client.c
+++ b/client.c
@@ -70,9 +70,10 @@ struct client_state {
    struct xdg_surface *xdg_surface;
    struct xdg_toplevel *xdg_toplevel;
    /* State */
-   bool closed;
    float offset;
    uint32_t last_frame;
+   int width, height;
+   bool closed;
};

    static void wl_buffer_release(void *data, struct wl_buffer
*wl_buffer) {
@@ -86,7 +87,7 @@ static const struct wl_buffer_listener
wl_buffer_listener = {
    static struct wl_buffer *
    draw_frame(struct client_state *state)
    {
-       const int width = 640, height = 480;
+       int width = state->width, height = state->height;
        int stride = width * 4;
        int size = stride * height;

@@ -124,6 +125,32 @@ draw_frame(struct client_state *state)
        return buffer;
    }

+static void
+xdg_toplevel_configure(void *data,
+                        struct xdg_toplevel *xdg_toplevel, int32_t width,
+                        int32_t height,
+                        struct wl_array *states)
+{
+    struct client_state *state = data;
+    if (width == 0 || height == 0) {
+        /* Compositor is deferring to us */
+        return;
+    }
+    state->width = width;
+    state->height = height;
+}
+
+static void
+xdg_toplevel_close(void *data, struct xdg_toplevel *toplevel)
+{
+    struct client_state *state = data;
+    state->closed = true;
+}
+
+static const struct xdg_toplevel_listener xdg_toplevel_listener = {
+    .configure = xdg_toplevel_configure,

```

```

+         .close = xdg_toplevel_close,
+     };
+
+     static void
+     xdg_surface_configure(void *data,
+                           struct xdg_surface *xdg_surface, uint32_t serial)
@@ -163,7 +190,7 @@ wl_surface_frame_done(void *data, struct
wl_callback *cb, uint32_t time)
+     {
+         cb = wl_surface_frame(state->wl_surface);
+         wl_callback_add_listener(cb, &wl_surface_frame_listener,
state);
+     }

-     /* Update scroll amount at 8 pixels per second */
+     /* Update scroll amount at 24 pixels per second */
+     if (state->last_frame != 0) {
+         int elapsed = time - state->last_frame;
+         state->offset += elapsed / 1000.0 * 24;
@@ -217,6 +244,8 @@ int
main(int argc, char *argv[])
+ {
+     struct client_state state = { 0 };
+     state.width = 640;
+     state.height = 480;
+     state.wl_display = wl_display_connect(NULL);
+     state.wl_registry = wl_display_get_registry(state.wl_display);
+     wl_registry_add_listener(state.wl_registry,
&wl_registry_listener, &state);
@@ -227,6 +256,8 @@ main(int argc, char *argv[])
+     state.xdg_wm_base, state.wl_surface);
+     xdg_surface_add_listener(state.xdg_surface,
&xdg_surface_listener, &state);
+     state.xdg_toplevel =
xdg_surface_get_toplevel(state.xdg_surface);
+     xdg_toplevel_add_listener(state.xdg_toplevel,
&xdg_toplevel_listener, &state);
+     xdg_toplevel_set_title(state.xdg_toplevel, "Example client");
+     wl_surface_commit(state.wl_surface);

```

如果你重新编译并运行这个客户端，你会发现它比之前规矩得多了。

请求状态变更

客户端也可以请求合成器把自己置于上述某个状态中，或者对窗口的尺寸施加约束。

```
<request name="set_max_size">
  <arg name="width" type="int"/>
  <arg name="height" type="int"/>
</request>

<request name="set_min_size">
  <arg name="width" type="int"/>
  <arg name="height" type="int"/>
</request>

<request name="set_maximized" />

<request name="unset_maximized" />

<request name="set_fullscreen" />
  <arg name="output"
    type="object"
    interface="wl_output"
    allow-null="true"/>
</request>

<request name="unset_fullscreen" />

<request name="set_minimized" />
```

合成器通过发送一个对应的 `configure` 事件来表示它对这些请求的确认。

1. 这会考虑客户端通过 `set_window_geometry` 请求发送的窗口几何信息。建议的尺寸只包含窗口几何所表示的空间。 [↩](#)

弹出窗口

在设计使用应用窗口的软件时，有许多场景会出于各种目的用到更小的次要表面。一些例子包括：右键时出现的上下文菜单、用于从若干选项中选取某个值的下拉框、鼠标悬停在某个 UI 元素上时显示的情境提示，以及窗口顶部和底部的菜单与工具栏。这些窗口常常会层层嵌套，例如沿着“File → Recent Documents → Example.odt”这样的路径打开。

对于 Wayland，XDG shell 提供了管理这些窗口的机制：弹出窗口。前面我们看过 `xdg_surface` 用于创建顶层应用窗口的“`get_toplevel`”请求。而对于弹出窗口，则改用“`get_popup`”请求。

```
<request name="get_popup">
  <arg name="id" type="new_id" interface="xdg_popup"/>
  <arg name="parent" type="object" interface="xdg_surface" allow-
null="true"/>
  <arg name="positioner" type="object" interface="xdg_positioner"/>
</request>
```

第一个和第二个参数的含义相当显而易见，但第三个参数引入了一个新概念：定位器。顾名思义，定位器的作用就是为新弹出窗口定位。它用于让合成器借助自己掌握的特权信息来参与弹出窗口的定位，例如避免让弹出窗口延伸到显示器边缘之外。我们将在第 10.4 节讨论定位器；眼下你只需创建一个定位器并把它传进去、不做任何进一步配置，就能借助相应的 `xdg_wm_base` 请求获得相当合理的默认行为：

```
<request name="create_positioner">
  <arg name="id" type="new_id" interface="xdg_positioner"/>
</request>
```

因此，简而言之，我们可以：

1. 创建一个新的 `wl_surface`
2. 为它获取一个 `xdg_surface`
3. 创建一个新的 `xdg_positioner`，把它的配置留到第 10.4 节
4. 从我们的 XDG 表面和 XDG 定位器创建一个 `xdg_popup`，并把它的父窗口指定为我们之前创建的 `xdg_toplevel`。

然后我们就可以用前面讨论过的同样生命周期，为弹出窗口表面渲染并附加缓冲区。我们还能使用一些其他专属于弹出窗口的特性。

配置

和 XDG 顶层窗口的 `configure` 事件一样，合成器有一个事件，可用来建议你的弹出窗口应当采用的尺寸。不过，与顶层窗口不同的是，它还会附带一个定位事件，把弹出窗口相对于其父表面的位置告知客户端。

```
<event name="configure">
  <arg name="x" type="int"
    summary="x position relative to parent surface window geometry"/>
  <arg name="y" type="int"
    summary="y position relative to parent surface window geometry"/>
  <arg name="width" type="int" summary="window geometry width"/>
  <arg name="height" type="int" summary="window geometry height"/>
</event>
```

客户端可以用 XDG 定位器来影响这些取值，这将在第 10.4 节讨论。

弹出窗口抓取

弹出窗口表面常常希望“抓取”（grab）所有输入，例如让用户用方向键选择不同的菜单项。这通过 grab 请求来实现：

```
<request name="grab">
  <arg name="seat" type="object" interface="wl_seat" />
  <arg name="serial" type="uint" />
</request>
```

发起这个请求的前提是已经收到了一个符合条件的输入事件，比如一次右键点击。该输入事件中的 `serial` 应当被用于这个请求。这些语义在第 9 章中有详细讨论。合成器之后可以取消这次抓取，例如当用户按下 `escape` 或点击你的弹出窗口外部时。

关闭

在合成器关闭你的弹出窗口的这些情形下，比如按下 `escape` 键时，会发送下面这个事件：

```
<event name="popup_done" />
```

为避免竞态条件，合成器会把弹出窗口结构保留在内存中，即便在它们被关闭之后也会处理针对它们的请求。关于对象生命周期和竞态条件的更多细节，参见第 2.4 节。

销毁弹出窗口

由客户端发起的弹出窗口销毁相当直接：

```
<request name="destroy" type="destructor" />
```

不过有一个细节值得一提：你必须自上而下地销毁所有弹出窗口。在任何时刻，你唯一能销毁的就是最顶层的那个。如果你不这么做，就会因协议错误而断开连接。

交互式移动与调整大小

许多应用窗口都带有可交互的 UI 元素，用户可以用它们来拖动或调整窗口大小。默认情况下，许多 Wayland 客户端都期望由自己负责窗口装饰，以提供这些可交互元素。在 X11 上，应用窗口可以在屏幕上的任意位置自行定位，并借此实现这些交互。

然而，Wayland 有一个刻意为之的设计特征：应用窗口并不知道自己在屏幕上的确切位置，也不知道自己相对于其他窗口的的位置。这一决定赋予了 Wayland 合成器更大的灵活性——窗口可以同时多个位置显示、被布置在 VR 场景的三维空间中，或者以任何其他新颖的方式呈现。Wayland 被设计成通用且广泛适用于众多设备与形态的协议。

为了平衡这两方面的设计需求，XDG 顶层窗口提供了两个请求，可以用来请求合成器开始一次交互式移动或调整大小操作。接口中相关的部分如下：

```
<request name="move">
  <arg name="seat" type="object" interface="wl_seat" />
  <arg name="serial" type="uint" />
</request>
```

与上一章介绍的弹出窗口创建请求类似，你必须提供一个输入事件 serial 才能启动交互式操作。例如，当你收到鼠标按键按下事件时，可以使用该事件的 serial 来开始一次交互式移动操作。接下来合成器会接手，在其内部坐标空间中为你的窗口开始一次交互式操作。

调整大小则稍微复杂一些，因为需要指定窗口的哪些边或角参与了该操作：

```
<enum name="resize_edge">
  <entry name="none" value="0"/>
  <entry name="top" value="1"/>
  <entry name="bottom" value="2"/>
  <entry name="left" value="4"/>
  <entry name="top_left" value="5"/>
  <entry name="bottom_left" value="6"/>
  <entry name="right" value="8"/>
  <entry name="top_right" value="9"/>
  <entry name="bottom_right" value="10"/>
</enum>

<request name="resize">
  <arg name="seat" type="object" interface="wl_seat" />
  <arg name="serial" type="uint" />
  <arg name="edges" type="uint" />
</request>
```

除此之外，它的工作方式大同小异。如果用户点击并沿着窗口左下角拖动，你可能希望发送一个交互式调整大小请求，带上相应的 `seat` 与 `serial`，并把 `edges` 参数设为 `bottom_left`。

还有一个请求，是客户端完整实现客户端侧窗口装饰所必需的：

```
<request name="show_window_menu">
  <arg name="seat" type="object" interface="wl_seat" />
  <arg name="serial" type="uint" />
  <arg name="x" type="int" />
  <arg name="y" type="int" />
</request>
```

当点击窗口装饰时，通常会弹出一个提供窗口操作（例如关闭或最小化窗口）的上下文菜单。对于由客户端管理窗口装饰的客户端来说，这个请求用于将客户端驱动的交互与合成器驱动的元素操作（例如最小化窗口）连接起来。如果你的客户端使用客户端侧装饰，你可以为此目的使用该请求。

xdg-decoration

在讨论客户端侧装饰的行为时，最后一个值得一提的细节，是决定是否使用它们的协商协议。不同的 Wayland 客户端与服务端可能对使用服务端侧还是客户端侧窗口装饰有着不同的偏好。为了表达这些意图，使用了一个协议扩展：`xdg-decoration`。它可以在 `wayland-protocols` 中找到。该协议提供了一个全局对象：

```
<interface name="zxdg_decoration_manager_v1" version="1">
  <request name="destroy" type="destructor" />

  <request name="get_toplevel_decoration">
    <arg name="id" type="new_id"
interface="zxdg_toplevel_decoration_v1"/>
    <arg name="toplevel" type="object" interface="xdg_toplevel"/>
  </request>
</interface>
```

你可以把 `xdg_toplevel` 对象传入 `get_toplevel_decoration` 请求，以获得一个具有如下接口的对象：

```
<interface name="zxdg_toplevel_decoration_v1" version="1">
  <request name="destroy" type="destructor" />

  <enum name="mode">
    <entry name="client_side" value="1" />
    <entry name="server_side" value="2" />
  </enum>

  <request name="set_mode">
    <arg name="mode" type="uint" enum="mode" />
  </request>

  <request name="unset_mode" />

  <event name="configure">
    <arg name="mode" type="uint" enum="mode" />
  </event>
</interface>
```

set_mode 请求用于表达客户端的偏好，而 unset_mode 用于表达无偏好。随后合成器会通过 configure 事件告知客户端是否使用客户端侧装饰。更多细节请查阅完整的 XML。

定位器

当我们几页之前介绍弹出窗口时，我们提到创建弹出窗口时必须提供一个定位器对象。我们当时请你别担心它、直接用默认值就好，因为它是个复杂的接口，而且与当时的主题无关。现在，我们将深入探究这个复杂的接口。

当你打开一个弹出窗口时，它会被显示在一个窗口系统中，而该窗口系统有一些你的客户端并不知情的约束。例如，Wayland 客户端并不知道自己的窗口显示在屏幕上的哪个位置。因此，如果你右键点击一个窗口，客户端并不掌握必要的信息去判断由此产生的弹出窗口是否会被推出屏幕边缘。定位器的设计正是为了解决这些问题：它允许客户端就弹出窗口如何移动或调整大小指定某些约束，然后由完全掌握实际情况的合成器来最终决定如何安置。

基础

```
<request name="destroy" type="destructor"></request>
```

这会在你用完后销毁定位器。你可以在弹出窗口创建之后再调用它。

```
<request name="set_size">
  <arg name="width" type="int" />
  <arg name="height" type="int" />
</request>
```

set_size 请求用于设置正在创建的弹出窗口的大小。

所有使用定位器的客户端都会用到这两个请求。现在，让我们进入有趣的部分。

锚定

剪贴板访问

数据提供对象

拖放

常用协议扩展

精确计时

指针约束

扩展剪贴板

桌面 shell 组件

杂项扩展

编写协议扩展

致谢

TODO（要感谢的人还有很多）